

Track C: Microsoft Fabric

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric	2
C1 Fabric architecture	3
C1-L1 Tenant, capacity, workspace and item	3
C1-L2 OneLake, one lake per tenant	6
C1-L3 Workspace roles, granted to groups	10
C2 Lakehouse, Warehouse and SQL analytics endpoint	15
C2-L1 Lakehouse Tables and Files, and schemas	15
C2-L2 Warehouse T-SQL, and where it differs from SQL Server.....	19
C2-L3 Lakehouse, Warehouse or SQL analytics endpoint.....	23
C3 Unifying datasets	29
C3-L1 OneLake shortcuts, and the delete that goes through	29
C3-L2 Mirroring operational databases into OneLake.....	32
C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric.....	36
C4 Ingestion and orchestration	41
C4-L1 Choosing how data gets into Fabric	41
C4-L2 Incremental copy with a watermark, or with CDC	45
C4-L3 Pipelines that fail fast, retry sensibly and alert.....	48
C5 Spark notebooks in Fabric	53
C5-L1 From Databricks notebooks to Fabric notebooks.....	53
C5-L2 Environments, resource profiles and V-Order.....	57
C5-L3 Delta tables shared between Databricks and Fabric	61
C6 Semantic models and Direct Lake	65
C6-L1 Star schemas for semantic models	65
C6-L2 DAX measures that state their filter context.....	69
C6-L3 Direct Lake, framing and the fallback you did not plan.....	72
Track C best-practice checklist	77
Appendix: solutions and answer keys	78

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C1 Fabric architecture

Roadmap: Months 16 to 18: MLOps and deployment; Current work.
Workspaces, capacities, items and OneLake as one logical lake.

C1-L1 Tenant, capacity, workspace and item

Concept

Everything in Fabric sits in one hierarchy. Learn it first, because cost, access and deployment all follow it.

- **Tenant.** Your organisation's Microsoft Entra ID tenant. Tenant settings, set by Fabric administrators, apply to everything below.
- **Capacity.** A pool of compute you buy. Every Fabric workload, from Spark to Power BI, draws from the capacity its workspace is assigned to.
- **Workspace.** A container where a team creates and shares items. Access is granted per workspace, through roles (lesson C1-L3).
- **Item.** A lakehouse, warehouse, notebook, pipeline, semantic model, report, eventhouse and so on. Items hold or process the data.

Capacities come as SKUs. F SKUs are the recommended Fabric capacities. The number is the compute in **capacity units** (CUs): an F2 has 2 CUs, an F64 has 64, up to F2048. A trial capacity has 64 CUs. Microsoft 365 P SKUs also run Fabric if Fabric is enabled on them.

Size affects licensing, not only speed. On **F64 or larger** (or a P capacity), users with a free Fabric license can view Power BI content. On smaller F SKUs, each viewer needs a Pro, Premium Per User or individual trial licence.

Pausing a capacity stops compute billing while it is paused. It is not free of consequences: when you pause, any smoothed and overage usage still owed is added to your Azure bill at once, and items on that capacity stop working until you resume.

Two practical rules follow from the hierarchy:

- Put dev, test and production in **separate workspaces**, and assign production to a capacity that experiments cannot exhaust.
- Before you design anything, find out which capacity SKU you have. It decides what users need to pay for and how much work runs at once.

Facts in this lesson come from Microsoft Learn's licensing and capacity pages, checked September 2026.

Why it matters

A team built a Power BI report for 300 frontline managers on an F32 capacity, assuming that managers with free licences could view it, as they had on the old Premium P1. Launch day produced 300 "You need a license" messages. The fix was either 300 Pro licences or a move to F64. Neither was in the budget, and the launch slipped a quarter.

Worked example: weak code

- | | |
|---|--|
| 1 | Workspace: Analytics (all projects) |
| 1 | Capacity: F32 shared with the data science sandbox |
| 2 | Viewers: 300 managers with free licences |
| 3 | Environments: dev and prod items in the same workspace |

1. **Line 2.** Production shares compute with experiments. One heavy notebook throttles the reports.
2. **Line 3.** Below F64, every viewer needs a Pro, PPU or trial licence.
3. **Line 4.** No separation: a change to a dev item can break production.

Worked example: strong code

- | | | |
|---|--|--------------------------------------|
| 1 | Workspace: sales_reporting_prod | -> capacity: F64 (production only) |
| 2 | Workspace: sales_reporting_test | -> capacity: F8 |
| 3 | Workspace: sales_reporting_dev | -> capacity: F8 |
| 4 | Viewers: 300 managers with free licences, reading the prod workspace's app | |
| 2 | Sandbox workspaces | -> capacity: F4, paused out of hours |

1. **Line 1.** F64 lets free-licence users view Power BI content, and production has its own compute.
2. **Line 5.** Experiments cannot starve production, and pausing saves money when nobody works.

The capacity decides who can view, how much runs at once and what it costs. Separate environments into workspaces, and give production its own capacity.

Recap card

- **Fabric hierarchy** Tenant, capacity, workspace, item.
- **What F64 means** An F SKU with 64 capacity units. F SKUs run from F2 to F2048.
- **Free-licence viewers of Power BI content** Need an F64 or larger (or P) capacity. Smaller F SKUs need Pro, PPU or trial licences for viewers.
- **Pausing a capacity** Stops compute billing from then on, bills remaining smoothed and overage usage at once, and stops its items.

Practice

Exercise 1. Multiple choice (level 1)

What does the number in the SKU name F32 tell you?

- a) 32 users can use the capacity.
- b) The capacity has 32 capacity units (CUs) of compute.
- c) 32 GB of OneLake storage is included.
- d) 32 workspaces can be assigned to it.

Why?

- r1) For F SKUs the number equals the capacity units: F2 is 2 CUs, F64 is 64.
- r2) SKU numbers are marketing names with no meaning.

Exercise 2. Put in order (level 2)

Order the Fabric hierarchy from the broadest to the most specific.

- Workspace
- Capacity
- Tenant
- Item, such as a lakehouse

Exercise 3. Architecture design (level 2)

A report will serve 200 viewers who have only free Fabric licences. Choose the set-up.

Options: F64 or larger, F16, cheaper, same features.

1. Capacity for the production workspace
2. What else runs on that capacity

Exercise 4. Write the function (level 3)

Write `cu_hours(sku: str, hours: float) -> float` for a cost estimate. The SKU is a string such as "F64". Return the capacity units multiplied by the hours running. Accept only the F SKUs F2, F4, F8, F16, F32, F64, F128, F256, F512, F1024 and F2048, and raise `ValueError` for any other SKU or for negative hours.

Starter code:

```
1 def cu_hours(sku, hours):  
2     return int(sku[1:]) * hours
```

Exercise 5. Multiple choice (level 4)

A team pauses its F64 capacity at 18:00 to save money after a heavy afternoon of Spark jobs. What happens?

- a) Nothing is billed for the afternoon's work, because the capacity is paused.
- b) Smoothed and overage usage still owed is added to the Azure bill at once, and items on the capacity stop working until it resumes.
- c) The capacity keeps running reports but stops Spark.
- d) OneLake data on the capacity is deleted after 24 hours.

Why?

- r1) Fabric smooths usage over time. Pausing settles what is still owed and makes the capacity's items unavailable.
- r2) Paused capacities are billed at double rate.

Sources

- F SKUs from F2 to F2048, where the number equals capacity units; a trial capacity has 64 CUs. (verified: <https://learn.microsoft.com/en-us/fabric/enterprise/licenses> (updated 2026-06-15))
- F64+ or P capacities support viewing Power BI content by users with a free Fabric licence; smaller F SKUs require Pro, PPU or trial licences for viewers. (verified: <https://learn.microsoft.com/en-us/fabric/enterprise/licenses> (updated 2026-06-15))
- When a capacity is paused, remaining cumulative overages and smoothed operations are added to the Azure bill. (verified: <https://learn.microsoft.com/en-us/fabric/enterprise/pause-resume> (updated 2026-04-20): pausing can prevent content from being available until resumed)

C1-L2 OneLake, one lake per tenant

Concept

Every Fabric tenant has exactly **one OneLake**. You cannot create a second one or delete it. It is built on Azure Data Lake Storage and stores tables in open formats: **Delta Parquet** or **Iceberg**. Every Fabric engine (Spark, the SQL engines, Power BI's Direct Lake, KQL) reads the same files, so one copy of the data serves them all.

OneLake mirrors the Fabric hierarchy: a folder per **workspace**, and inside it a folder per **item**, named with its type. A lakehouse holds two areas: **Tables** for managed Delta tables and **Files** for anything else.

You address data with an ADLS-style path:

```
1 https://onelake.dfs.fabric.microsoft.com/<workspace>/<item>.<itemtype>/<path>
2 abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/<path>
```

For example `abfss://sales_prod@onelake.dfs.fabric.microsoft.com/lh_sales.lakehouse/Tables/orders`. The item type is lower case: `.lakehouse`, `.warehouse`.

Names change. **GUIDs do not**. OneLake accepts the workspace and item GUIDs in place of names, and they stay fixed when someone renames the workspace or item. With GUIDs you give both the workspace and the item as GUIDs, and you drop the item type. You find them in the item's URL in the Fabric portal.

Use GUIDs in pipelines, jobs and code that other people will run. Two more reasons from Microsoft Learn:

- The `abfs` driver URI does not allow special characters such as spaces in the workspace name. GUIDs avoid the problem.
- The global endpoint `onelake.dfs.fabric.microsoft.com` can route through another region. If data residency matters, use your workspace's regional endpoint, `https://<region>-onelake.dfs.fabric.microsoft.com`.

Because tools that speak ADLS APIs can read OneLake, Azure Databricks, Azure Storage Explorer or Python's Azure SDK can read Fabric data with the right permissions.

Why it matters

A Databricks job read a Fabric lakehouse through `abfss://Sales Analytics@onelake.dfs.fabric.microsoft.com/...`. It failed on the first run, because the workspace name held a space. The engineer renamed the workspace to `SalesAnalytics`. The job worked, and four Power BI dataflows and two pipelines that referred to the old name broke that night. Using the workspace and item GUIDs would have avoided both failures.

Worked example: weak code

```
❶ path = "abfss://Sales Analytics@onelake.dfs.fabric.microsoft.com/Sales
        LH.Lakehouse/Tables/orders"
```

```
2 df = spark.read.format("delta").load(path)
```

1. **Line 1.** Spaces break the abfs URI, the item type is not lower case, and a rename breaks the path.

Worked example: strong code

```
1 ONELAKE_HOST = "onelake.dfs.fabric.microsoft.com"
2
3
4 def onelake_table_path(workspace_id: str, item_id: str, table: str) ->
    str:
5     """abfss path to a lakehouse table, by GUID, stable across renames."""
6     1 return f"abfss://{workspace_id}@{ONELAKE_HOST}/{item_id}/Tables/
    {table}"
7
8
9     2 path = onelake_table_path(CONFIG["workspace_id"], CONFIG["lakehouse_id"],
    "orders")
10 df = spark.read.format("delta").load(path)
```

1. **Line 6.** With GUIDs, both workspace and item are IDs and the item type is dropped.
2. **Line 9.** IDs come from config, not from code (lesson A1-L3).

One lake, one copy, many engines. Address data by GUID in anything automated.

Recap card

- **How many OneLakes per tenant?** Exactly one. You cannot create or delete OneLakes.
- **OneLake table formats** Delta Parquet or Iceberg, open formats every Fabric engine reads.
- **OneLake path by name** abfss://
<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<itemtype>/
Tables/<table>.
- **Why GUIDs in automation** They survive renames, avoid special-character problems in abfs URIs, and drop the item type.

Practice

Exercise 1. Multiple choice (level 1)

Your company has three business units in one Fabric tenant. How many OneLakes are there?

- a) One per business unit

- b) One for the whole tenant
- c) One per capacity
- d) One per workspace

Why?

- r1) Every tenant has a single OneLake. Workspaces organise it; they do not create new lakes.
- r2) OneLake licences are sold per tenant.

Exercise 2. Fill the gap (level 2)

Complete the path to table `orders` in lakehouse `lh_sales`, workspace `sales_prod`.

```
1 abfss://____(1)@onelake.dfs.fabric.microsoft.com/lh_sales.____(2)/
   ____ (3)/orders
```

Exercise 3. Spot the bug (level 3)

This path fails with an invalid URI error. Tap the line with the problem and rewrite the path so it survives renames too.

```
1 WORKSPACE = "Sales Analytics"
2 LAKEHOUSE = "lh_sales"
3 path = f"abfss://{WORKSPACE}@onelake.dfs.fabric.microsoft.com/
   {LAKEHOUSE}.lakehouse/Tables/orders"
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write `onelake_path(workspace: str, item: str, item_type: str, relative: str) -> str` returning `abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.<item_type>/<relative>`.

- `item_type` must be `"lakehouse"` or `"warehouse"`. Otherwise raise `ValueError`.
- Raise `ValueError` if `workspace` or `item` contains a space.
- Strip leading slashes from `relative`.

Starter code:

```
1 def onelake_path(workspace, item, item_type, relative):
2     return f"abfss://{workspace}@onelake.dfs.fabric.microsoft.com/{item}.
   {item_type}/{relative}"
```

Exercise 5. Architecture design (level 4)

An Azure Databricks job in the UK will read a Fabric lakehouse in a UK South workspace every night. Data must stay in the UK. Choose the set-up.

Options: GUIDs from config, Display names in the code.

- 1. How the job refers to workspace and lakehouse
- 2. Endpoint
- 3. Data movement

Sources

- Every tenant has a single OneLake, built on ADLS, storing tables in Delta Parquet or Iceberg; you cannot create multiple OneLakes. (verified: <https://learn.microsoft.com/en-us/fabric/onelake/onelake-overview> (updated 2026-03-09))
- OneLake URI formats by name and by GUID; GUIDs do not change on rename; abfs URIs do not allow special characters such as spaces in the workspace name; the global endpoint may route data out of region, and regional endpoints avoid this. (verified: <https://learn.microsoft.com/en-us/fabric/onelake/onelake-access-api> (updated 2026-05-18))

C1-L3 Workspace roles, granted to groups

Concept

A workspace has four roles: **Admin**, **Member**, **Contributor** and **Viewer**. Each role includes the rights of the ones below it, with a few rights reserved for the top.

From Microsoft Learn's role table (checked September 2026):

Right	Admin	Member	Contributor	Viewer
Update or delete the workspace, add or remove admins	yes			
Connect the workspace to Git, create a workspace identity	yes			
Add members or lower roles, allow resharing	yes	yes		
Create and change items, run notebooks and pipelines	yes	yes	yes	
Read lakehouse and warehouse data through Spark and OneLake APIs (ReadAll)	yes	yes	yes	
	yes	yes	yes	yes

Right	Admin	Member	Contributor	Viewer
Read lakehouse and warehouse data with T-SQL (ReadData)				

Two consequences surprise people coming from Databricks:

- A **Viewer** can query a lakehouse through its SQL analytics endpoint, but cannot read the same tables from a Spark notebook or through OneLake APIs.
- A **Contributor** can change and delete every item in the workspace, including production lakehouse tables. Contributor in production is powerful.

Rules that keep access manageable:

- **Assign roles to Microsoft Entra security groups**, not to individuals (as in lesson A4-L2). People join and leave groups. Role assignments stay the same.
- **Least privilege per environment.** Engineers are Contributors in dev. In production, deployment runs under an automation identity, and people are Viewers unless they are on call.
- **Keep at least two admins**, as groups or named people, so a departure does not lock the workspace.
- Workspace roles are coarse. For finer control inside an item, use item sharing, SQL permissions, or OneLake security (lesson C9-L2).

Why it matters

A contractor was added to a production workspace as Contributor "to look at a notebook". Two weeks later they ran a clean-up notebook against the wrong lakehouse and dropped three tables that reports depended on. Viewer would have let them read the notebook and query the data with T-SQL, and would not have let them change anything.

Worked example: weak code

```

1 workspace: sales_prod
2 roles:
3   Admin: [sam@contoso.com]
4   Contributor: [grp-sales-analysts, contractor@partner.com]
5   Viewer: []

```

1. **Line 3.** One named admin: if Sam leaves, nobody can manage the workspace.
2. **Line 4.** Write access to production for people who need to read, including an external individual.

Worked example: strong code

```
1 workspace: sales_prod
2 roles:
3   ① Admin: [grp-fabric-platform-admins]
4   ② Member: [grp-sales-data-leads]
5   ③ Contributor: [sp-sales-deployment]
6   ④ Viewer: [grp-sales-analysts, grp-sales-contractors]
```

1. **Line 3.** A group of admins: no single point of failure.
2. **Line 5.** Only the deployment identity writes to production.
3. **Line 6.** Readers query with T-SQL and reports, and cannot change items.

Roles go to groups, write access in production goes to automation, and people read.

Recap card

- **Fabric workspace roles** Admin, Member, Contributor, Viewer.
- **Admin-only rights** Update or delete the workspace, manage admins, connect Git, create a workspace identity.
- **Viewer and lakehouse data** Reads with T-SQL (ReadData). Not through Spark or OneLake APIs (ReadAll starts at Contributor).
- **Who writes to production?** An automation identity as Contributor. People get Viewer, through groups.

Practice

Exercise 1. Multiple choice (level 1)

Which workspace role can connect a workspace to a Git repository?

- a) Admin only
- b) Admin and Member
- c) Any role except Viewer
- d) Any role

Why?

- r1) Connecting Git, like deleting the workspace or creating a workspace identity, is reserved for Admins.
- r2) Git is a paid feature for Admins.

Exercise 2. Multiple choice (level 2)

An analyst with the Viewer role runs `spark.read.table("lh_sales.orders")` in a notebook in that workspace. What happens?

- a) It works: Viewers read everything.
- b) It fails: Viewers read lakehouse data with T-SQL, not through Spark or OneLake APIs.
- c) It works, but returns only the first 1,000 rows.
- d) It fails because Viewers cannot see notebooks at all.

Why?

- r1) Viewer includes ReadData (T-SQL) but not ReadAll (Spark and OneLake APIs).
- r2) Spark is disabled for all users in production workspaces.

Exercise 3. Architecture design (level 2)

Assign roles in the production workspace `sales_prod`.

Options: Admin, Member, Contributor, Viewer.

1. Fabric platform team (group)
2. Deployment service principal
3. Sales analysts (group)

Exercise 4. Write the function (level 3)

Write `min_role(action: str) -> str` returning the lowest workspace role that allows the action. Use these actions from the role table:

- `"connect_git"`, `"delete_workspace"`: Admin
- `"add_members"`: Member
- `"run_notebook"`, `"spark_read"`: Contributor
- `"tsql_read"`: Viewer

Raise `KeyError` for any other action.

Starter code:

```
1 def min_role(action):
2     return "Viewer"
```

Exercise 5. Spot the bug (level 4)

This role file for the production workspace breaks two rules of the lesson. Tap the line with the more dangerous one, then rewrite the file.

```
1 workspace: sales_prod
```

```
2 roles:
3   Admin: [grp-fabric-platform-admins]
4   Contributor: [sp-sales-deployment, grp-sales-analysts]
5   Viewer: [jo.bloggs@contoso.com]
```

Write down the line number of the bug. Then write the fix.

Sources

- The four workspace roles and their rights, including Admin-only Git connection and Viewer having ReadData but not ReadAll. (verified: <https://learn.microsoft.com/en-us/fabric/fundamentals/roles-workspaces> (updated 2026-04-24))

C2 Lakehouse, Warehouse and SQL analytics endpoint

Roadmap: Months 16 to 18: MLOps and deployment; Current work. When to use each.

C2-L1 Lakehouse Tables and Files, and schemas

Concept

A lakehouse has two top-level areas in OneLake:

- **Tables** holds **managed Delta tables**. Fabric registers them, the SQL analytics endpoint exposes them, and Direct Lake semantic models read them.
- **Files** holds anything else: CSV, JSON, Parquet, images, raw deliveries.

Only Delta tables in Tables reach the SQL engines. A Parquet or CSV file is invisible to the SQL analytics endpoint, wherever it sits. If you put a folder that is not a valid Delta table under Tables, the lakehouse explorer shows it in an **Unidentified** area. A broken shortcut also lands there.

So the flow for a delivery is: land the raw file in **Files** (your bronze copy, as received), then read it with Spark and write a Delta table to **Tables** with `saveAsTable`.

Schemas. New lakehouses created in the portal have schemas enabled by default. Every schema-enabled lakehouse has a `dbo` schema that you cannot rename or remove. A table written without a schema goes to `dbo`. Create schemas by domain (`sales`, `hr`) to keep a growing lakehouse navigable.

In Spark SQL you reference tables with a **four-part name**:

`workspace.lakehouse.schema.table`. Parts you leave out come from defaults: a bare table name means the `dbo` schema of the notebook's default lakehouse. For code that others run, name at least the lakehouse and schema, so it does not depend on which lakehouse happens to be attached.

One change to note if you learned Fabric earlier: since 5 September 2025, Fabric no longer creates a default semantic model automatically with each new lakehouse. Create semantic models on purpose (lesson C6-L1).

Why it matters

An engineer wrote cleaned survey data as Parquet into `Files/clean/respondents/`, the way they had written to cloud storage before. The Power BI team could not find the table in the SQL analytics endpoint, so they imported the Parquet into a separate dataflow, a second copy with its own refresh. Writing a Delta table to Tables with `saveAsTable` would have exposed it to every engine with no copy.

Worked example: weak code

```
❶ raw = spark.read.option("header", True).csv("Files/landing/
    respondents_w14.csv")
❷ raw.write.mode("overwrite").parquet("Files/clean/respondents")
```

1. **Line 1.** Every column read as a string, types unchecked.
2. **Line 2.** Parquet under Files: invisible to the SQL analytics endpoint and Direct Lake.

Worked example: strong code

```
1 from pyspark.sql import DataFrame, SparkSession
2 from pyspark.sql import types as T
3
4 RESPONDENT_SCHEMA = T.StructType([
5     T.StructField("respondent_id", T.StringType(), False),
6     T.StructField("wave", T.IntegerType(), False),
7     T.StructField("weight", T.DoubleType(), True),
8 ])
9
10
11 def load_delivery(spark: SparkSession, path: str, table: str) ->
    DataFrame:
12     """Read a CSV delivery from Files and append it to a managed Delta
        table in Tables."""
    ❶ df = spark.read.option("header",
        True).schema(RESPONDENT_SCHEMA).csv(path)
    ❷ df.write.mode("append").saveAsTable(table)
15     return df
16
17
    ❸ load_delivery(spark, "Files/landing/respondents_w14.csv",
        "lh_survey.sales.respondents")
```

1. **Line 13.** An explicit schema: types are decided, not inferred.
2. **Line 14.** A managed Delta table in Tables: visible to Spark, T-SQL and Direct Lake.
3. **Line 18.** Lakehouse and schema named, so the code does not depend on the attached default.

Raw files in Files, Delta tables in Tables, and schema-qualified names in shared code.

Recap card

- **Lakehouse Tables vs Files** Tables: managed Delta tables, visible to SQL and Direct Lake. Files: raw and non-Delta data.
- **Non-Delta folder under Tables** Shown in the Unidentified area, not queryable.
- **Lakehouse schemas** On by default for new lakehouses. dbo is the fixed default schema.
- **Spark four-part name** workspace.lakehouse.schema.table. A bare name means dbo in the default lakehouse.

Practice

Exercise 1. Multiple choice (level 1)

A Parquet file sits at `Files/exports/orders.parquet` in a lakehouse. What can the SQL analytics endpoint do with it?

- a) Query it as a table named orders.
- b) Nothing. Only Delta tables in Tables appear in the endpoint.
- c) Query it after a metadata refresh.
- d) Convert it to Delta automatically.

Why?

- r1) The endpoint exposes Delta tables only. Parquet and CSV cannot be queried through it.
- r2) Files is a private folder only the owner can see.

Exercise 2. Multiple choice (level 2)

In a notebook attached to schema-enabled lakehouse `lh_survey`, you run `spark.read.table("respondents")`. Which table does Spark read?

- a) `lh_survey.dbo.respondents`
- b) The first schema that has a respondents table
- c) It raises an error: schemas require a four-part name
- d) respondents in every schema, unioned

Why?

- r1) A bare table name resolves to the dbo schema of the default lakehouse.

- r2) dbo is the only schema Spark can read.

Exercise 3. Fill the gap (level 2)

Join two tables in different workspaces in one Spark SQL query.

```
1 SELECT e.name, d.dept_name
2 FROM ops_ws.hr_lakehouse.____(1).employees AS e
3 JOIN global_ws.corporate_lakehouse.company.departments AS d
4   ON e.deptno = d.deptno
```

Exercise 4. Write the function (level 3)

Write `load_delivery(spark, path, table)`. Read the CSV at `path` with a header and the given `RESPONDENT_SCHEMA`, append it to the managed table `table` with `saveAsTable`, and return the DataFrame you read. Do not write Parquet to Files.

Starter code:

```
1 from pyspark.sql import types as T
2
3 RESPONDENT_SCHEMA = T.StructType([
4     T.StructField("respondent_id", T.StringType(), False),
5     T.StructField("wave", T.IntegerType(), False),
6     T.StructField("weight", T.DoubleType(), True),
7 ])
8
9
10 def load_delivery(spark, path, table):
11     df = spark.read.option("header", True).csv(path)
12     df.write.mode("overwrite").parquet("Files/clean/respondents")
13     return df
```

Exercise 5. Architecture design (level 4)

Lay out a lakehouse for a survey programme with raw CSV deliveries, cleaned respondent data and reporting tables.

Options: Files, Tables, dbo schema, Tables, a named schema per domain.

1. Raw CSV deliveries, kept as received
2. Cleaned respondent-level data
3. Reporting tables read by Power BI

Sources

- Lakehouse Tables holds managed Delta tables and Files holds unstructured or non-Delta data; only Delta tables appear in the SQL analytics endpoint; default semantic models are no longer created automatically since 5 September 2025. (verified: <https://>)

learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-overview (updated 2026-05-07))

- Non-Delta folders under Tables and broken shortcuts appear in the Unidentified area. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/navigate-lakehouse-explorer> (updated 2026-05-07))
- Schemas are enabled by default for new lakehouses; dbo cannot be renamed or removed; Spark uses workspace.lakehouse.schema.table and resolves omitted parts from defaults. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-schemas> (updated 2026-02-24))

C2-L2 Warehouse T-SQL, and where it differs from SQL Server

Concept

Fabric Warehouse speaks T-SQL, and most SQL Server habits carry over:

`CREATE TABLE`, `INSERT`, `UPDATE`, `DELETE`, stored procedures, views, `TRUNCATE TABLE`, identity columns and explicit transactions. `MERGE` is generally available. Its tables are Delta tables in OneLake, so Spark and Power BI read them too.

The differences cause real bugs. Microsoft Learn, checked September 2026:

Keys are not enforced. `PRIMARY KEY` is supported only as `NONCLUSTERED NOT ENFORCED`. `UNIQUE` likewise, and `FOREIGN KEY` only `NOT ENFORCED`. The engine uses them as hints for query plans. It does **not** stop duplicates or orphans. Deduplicate and check keys yourself (lesson A3-L1 for `MERGE` sources, lesson A6-L1 for contracts).

Some data types are missing, with documented alternatives:

Not supported	Use
<code>nchar</code> , <code>nvarchar</code>	<code>char</code> , <code>varchar</code> (UTF-8 collation)
<code>money</code> , <code>smallmoney</code>	<code>decimal</code>
<code>datetime</code> , <code>smalldatetime</code>	<code>datetime2</code> (up to 6 fractional digits)
<code>tinyint</code>	<code>smallint</code>
<code>text</code> , <code>ntext</code> , <code>json</code>	<code>varchar</code>

`varchar(max)` is limited to 16 MB per value.

Some features are absent or in preview. Triggers are not supported.

`ALTER COLUMN` and nested CTEs are in preview. Check the T-SQL surface area page before you port a procedure that relies on them.

Porting a SQL Server script is usually a type-mapping job plus one design change: replace constraint enforcement with explicit checks.

Why it matters

A team ported a customer dimension from SQL Server, where `PRIMARY KEY` had rejected duplicate loads for years. In Fabric the same key was created `NOT ENFORCED`, so a pipeline that ran twice on the same file loaded every customer twice. Sales per customer halved in the joined reports for a week before anyone traced it. A duplicate check after each load would have failed the run on the first rerun.

Worked example: weak code

```
1 CREATE TABLE dbo.dim_customer (  
  ① customer_id INT PRIMARY KEY,  
  ② name NVARCHAR(200),  
  ③ credit_limit MONEY,  
  ④ created_at DATETIME  
6 );
```

1. **Line 2.** An enforced clustered primary key is not supported in Fabric Warehouse.
2. **Line 3.** `nvarchar` is not supported. Use `varchar` with the UTF-8 collation.
3. **Line 4.** `money` is not supported. Use `decimal`.
4. **Line 5.** `datetime` is not supported. Use `datetime2`.

Worked example: strong code

```
1 CREATE TABLE dbo.dim_customer (  
2   customer_id INT NOT NULL,  
3   name VARCHAR(200),  
  ① credit_limit DECIMAL(19, 4),  
4   created_at DATETIME2(6)  
5 );  
6  
7 ALTER TABLE dbo.dim_customer  
  ② ADD CONSTRAINT pk_dim_customer PRIMARY KEY NONCLUSTERED (customer_id)  
    NOT ENFORCED;  
8  
9  
10 -- The key is a hint, so check it after every load.  
11 SELECT customer_id, COUNT(*) AS n  
12 FROM dbo.dim_customer  
13 GROUP BY customer_id  
  ③ HAVING COUNT(*) > 1;
```

1. **Line 4.** `decimal` replaces `money`, with the scale stated.

2. **Line 8.** The only supported form of primary key: a hint for the optimiser, not a guard.
3. **Line 14.** The duplicate check the key no longer performs. Any row returned fails the load.

Map the types, declare keys as NOT ENFORCED hints, and check uniqueness yourself after every load.

Recap card

- **Primary keys in Fabric Warehouse** Only PRIMARY KEY NONCLUSTERED ... NOT ENFORCED. Duplicates are not blocked.
- **nvarchar, money, datetime in Fabric Warehouse** Not supported. Use varchar, decimal, datetime2.
- **MERGE and TRUNCATE in Fabric Warehouse** Both supported. MERGE is generally available.
- **Triggers in Fabric Warehouse** Not supported. Put the logic in the loading procedure.

Practice

Exercise 1. Multiple choice (level 1)

A Fabric Warehouse table has PRIMARY KEY NONCLUSTERED (customer_id) NOT ENFORCED. You insert two rows with customer_id = 7. What happens?

- a) The second insert fails with a key violation.
- b) Both rows are stored. The key is not enforced.
- c) The second row replaces the first.
- d) The table is locked until you remove a duplicate.

Why?

- r1) NOT ENFORCED keys are optimiser hints. The engine does not check them.
- r2) Fabric deduplicates later in a background job.

Exercise 2. Fill the gap (level 2)

Add the only supported form of primary key to dbo.dim_customer.

```
1 ALTER TABLE dbo.dim_customer
2     ADD CONSTRAINT pk_dim_customer PRIMARY KEY ____ (1) (customer_id)
   ____ (2);
```

Exercise 3. Spot the bug (level 3)

This table definition, ported from SQL Server, fails in Fabric Warehouse. Tap one failing line, then rewrite the definition with supported types.

```
1 CREATE TABLE dbo.fact_payment (  
2     payment_id BIGINT NOT NULL,  
3     customer_id INT NOT NULL,  
4     amount MONEY NOT NULL,  
5     note NVARCHAR(500),  
6     paid_at DATETIME2(6) NOT NULL  
7 );
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write a query that returns every `customer_id` appearing more than once in `dbo.dim_customer`, with its count as `n`. The load job fails if it returns any row.

Starter code:

```
1 SELECT DISTINCT customer_id FROM dbo.dim_customer;
```

Exercise 5. Architecture design (level 4)

You port a SQL Server data mart to Fabric Warehouse. Decide how to handle each feature.

Options: NOT ENFORCED keys plus an explicit duplicate check after each load, NOT ENFORCED keys, trusting the source.

1. Primary keys that used to block duplicates
2. An audit trigger on the orders table
3. nvarchar and money columns

Sources

- Identity columns, TRUNCATE TABLE and MERGE (GA) are supported; triggers are not; ALTER COLUMN and nested CTEs are in preview. (verified: <https://learn.microsoft.com/en-us/fabric/data-warehouse/tsql-surface-area> (updated 2026-08-26))
- PRIMARY KEY and UNIQUE are supported only as NONCLUSTERED NOT ENFORCED, FOREIGN KEY only as NOT ENFORCED. (verified: <https://learn.microsoft.com/en-us/fabric/data-warehouse/table-constraints> (updated 2026-09-08))
- Unsupported types and alternatives: nchar/nvarchar to char/varchar, money to decimal, datetime to datetime2, tinyint to smallint; varchar(max) limited to 16 MB; datetime2 limited to 6 fractional digits.

(verified: <https://learn.microsoft.com/en-us/fabric/data-warehouse/data-types> (updated 2026-06-11))

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

Concept

Fabric gives you three ways to hold and query tables. All three store data as Delta tables in OneLake. They differ in who writes the data and in which language.

Lakehouse. You write with Spark: notebooks, Spark job definitions, Dataflows Gen2 or pipelines. It holds files and Delta tables side by side. Choose it for raw files, PySpark transformations and data science. This is the closest match to your Databricks work.

SQL analytics endpoint. Every lakehouse gets one automatically. It gives a **read-only** T-SQL surface over the lakehouse's Delta tables. You can `SELECT`, and you can create views, functions and stored procedures. You cannot `INSERT`, `UPDATE` or `DELETE` table data through it. It picks up new or changed tables through a metadata sync. Microsoft documents the usual lag as under one minute, but it varies.

Warehouse. You write with T-SQL. It supports full DML and DDL, and multi-table transactions. Choose it when a team works in T-SQL, loads data with stored procedures, or needs several tables to change together or not at all.

One rule decides most cases: **who writes the data, and in which language?** Spark writers go to a lakehouse. T-SQL writers go to a warehouse. T-SQL readers of lakehouse data use the SQL analytics endpoint.

You do not have to copy data between them. A warehouse query reads lakehouse tables in the same workspace with a three-part name:

`lakehouse_name.schema.table`. Direct Lake semantic models read from either.

Check the status of each feature you rely on in `VERSIONS.md` before you build. Fabric changes monthly.

Why it matters

A team moves a finance reconciliation from SQL Server to Fabric. They land every table in a lakehouse, because the Databricks team already uses lakehouses. The month-end stored procedure updates three tables. On the SQL analytics endpoint it fails on its first `UPDATE`, because the endpoint is read-only.

Under deadline, someone rewrites it as three separate Spark notebook writes. On one month-end the second notebook fails. Table one holds the new month and table three holds the old one, and the reconciliation report shows a balance that does not tie. A warehouse transaction would have rolled back all three tables together.

Worked example: weak code

```
❶ -- Connected to the SQL analytics endpoint of lakehouse lh_finance
2 CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3 BEGIN
❷ UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
5 INSERT INTO dbo.balances
6 SELECT account_id, @period, SUM(amount) FROM dbo.ledger
7 WHERE period = @period GROUP BY account_id;
❸ DELETE FROM dbo.open_items WHERE period = @period;
9 END;
```

1. **Line 1.** The endpoint gives read-only access to lakehouse tables. Every write below fails.
2. **Line 4.** UPDATE, INSERT and DELETE on lakehouse tables are not supported through the endpoint.
3. **Line 8.** Three tables must change together. Without a transaction, a failure halfway leaves them inconsistent.

Worked example: strong code

```
❶ -- Connected to warehouse wh_finance
2 CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3 BEGIN
❷ BEGIN TRANSACTION;
5 UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
❸ INSERT INTO dbo.balances (account_id, period, balance)
7 SELECT account_id, @period, SUM(amount) FROM dbo.ledger
8 WHERE period = @period GROUP BY account_id;
9 DELETE FROM dbo.open_items WHERE period = @period;
10 COMMIT TRANSACTION;
11 END;
```

1. **Line 1.** The T-SQL writer works in a warehouse, which supports full DML.
2. **Line 4.** All three changes commit together or not at all. Warehouses support multi-table transactions.
3. **Line 6.** Name the target columns. An INSERT without a column list breaks when someone adds a column.

The data and the logic are the same. The item type decides whether the procedure runs at all, and whether a failure leaves the tables consistent.

Recap card

- **Lakehouse, Warehouse or endpoint: the deciding question** Who writes the data, and in which language? Spark writes to a lakehouse, T-SQL writes to a warehouse.
- **What can you do on the SQL analytics endpoint?** SELECT and create views, functions and procedures. No INSERT, UPDATE or DELETE on table data.
- **When do you need a Warehouse over a Lakehouse?** T-SQL writes, stored procedure loads, and multi-table transactions.
- **Read a lakehouse table from a warehouse in the same workspace** Use a three-part name: lakehouse.schema.table. No copy.
- **A new lakehouse table is missing from the endpoint** Metadata sync lag. Usually under a minute. Refresh or retry before dependent steps.

Practice

Exercise 1. Multiple choice (level 1)

Which statement about a lakehouse's SQL analytics endpoint is true?

- a) You can run UPDATE on lakehouse tables through it.
- b) You can create a view over lakehouse tables in it.
- c) You create it by hand after you create the lakehouse.
- d) It stores its own copy of the lakehouse tables.

Why?

- r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.
- r2) Views are stored inside the Delta tables.

Exercise 2. Architecture design (level 2)

Pick the Fabric item for each workload.

Options: Lakehouse, Warehouse, SQL analytics endpoint.

1. Land raw JSON survey deliveries and clean them with PySpark
2. Finance loads budgets with T-SQL stored procedures that change several tables together
3. Analysts query the cleaned lakehouse tables with T-SQL and save a few views

Exercise 3. Spot the bug (level 3)

This script runs against the SQL analytics endpoint of `lh_research`. It fails partway. Tap the failing line, then rewrite the script so it runs on the endpoint.

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W00';
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

On the SQL analytics endpoint, write a view `dbo.v_wave_reach` with one row per `wave_id` and a column `reach_pct`: the sum of `weighted_reach` divided by the sum of `weighted_base` from `dbo.nat_metrics`. A wave with a zero base must return NULL, not a divide-by-zero error.

Starter code:

```
1 CREATE VIEW dbo.v_wave_reach AS
```

Exercise 5. Fill the gap (level 4)

You are connected to warehouse `wh_finance`. Lakehouse `lh_research` sits in the same workspace. Complete the query so it reads the lakehouse table without copying it.

```
1 INSERT INTO dbo.wave_summary (wave_id, weighted_base)
2 SELECT wave_id, SUM(weighted_base)
3 FROM _____.(1).dbo.nat_metrics
4 GROUP BY wave_id;
```

Exercise 6. Multiple choice (level 5)

A pipeline runs a Spark notebook that creates a new gold table in `lh_research`. The next pipeline step immediately queries the table through the SQL analytics endpoint and fails with "Invalid object name". An hour later the same query works. What is the most likely cause?

- a) The notebook wrote the table in a format the endpoint cannot read.
- b) The endpoint had not yet synced the new table's metadata.
- c) The endpoint needs a manual refresh every time a table changes.
- d) The pipeline's identity lacks permission on new tables for one hour.

Why?

- r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.
- r2) The endpoint copies data from the lakehouse on a schedule.

Practical: Lakehouse, endpoint and warehouse side by side

Platform: Fabric trial workspace. Run this in a Fabric trial workspace. You create a lakehouse and a warehouse, write one table with Spark, read it through the endpoint, and see a write fail there.

1. In your trial workspace, create a lakehouse named `lh_research`.
2. Open a new notebook attached to `lh_research`. Run:

```
spark.createDataFrame([("W01", 120.0, 400.0), ("W02", 0.0, 0.0)],  
"wave_id STRING, weighted_reach DOUBLE, weighted_base  
DOUBLE").write.mode("overwrite").saveAsTable("nat_metrics").
```
3. Switch the lakehouse view to its SQL analytics endpoint. Wait for `nat_metrics` to appear. Note how long it takes.
4. Open a new SQL query on the endpoint and create the `dbo.v_wave_reach` view from exercise 4. Query it. Wave W02 returns NULL.
5. In the same query window, run `UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W01'`; and read the error message.
6. Create a warehouse named `wh_finance`. In a new query, run `SELECT * FROM lh_research.dbo.nat_metrics;`. It returns the rows without a copy.
7. Write down what you saw in each step. Compare it with the recap card.

Sources

- The SQL analytics endpoint is read-only for Delta table data. No INSERT, UPDATE or DELETE. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint> (updated 2026-05-19))
- You can create views, functions and stored procedures in the SQL analytics endpoint. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint> (updated 2026-05-19))
- Fabric Warehouse supports explicit multi-table transactions with BEGIN TRAN, COMMIT TRAN and ROLLBACK TRAN. (verified: <https://learn.microsoft.com/en-us/fabric/data-warehouse/transactions> (updated 2026-06-03))
- Metadata sync runs in the background, and you can refresh it from the portal, the REST API or `sys.sp_dw_refresh_ext_table`. (verified: <https://>

learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-metadata-sync (updated 2026-05-29))

- A warehouse reads lakehouse tables in the same workspace with three-part names, and transactions can span them. (verified: <https://learn.microsoft.com/en-us/fabric/data-warehouse/query-warehouse> and .../transactions)
- Under normal conditions the lag between a lakehouse and its SQL analytics endpoint is under one minute; the background sync stops after 15 minutes without query activity. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-performance> (updated 2026-08-07))

C3 Unifying datasets

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Shortcuts, mirroring (including Azure Databricks Unity Catalog) and avoiding duplicate copies.

C3-L1 OneLake shortcuts, and the delete that goes through

Concept

A **shortcut** is a folder in a lakehouse (or KQL database) that points to data somewhere else. It behaves like a symbolic link. The target can be inside OneLake, such as a table in another workspace, or outside it: ADLS Gen2, Azure Blob Storage, Amazon S3 and S3-compatible storage, Google Cloud Storage, Dataverse, OneDrive or SharePoint.

Shortcuts let every Fabric engine read the data **without copying it**. OneLake handles the connection and credentials, so each workload does not need its own.

Where you create the shortcut matters:

- Under **Tables**, point it at Delta data. The lakehouse recognises the folder as a table, and the SQL analytics endpoint and Direct Lake can use it.
- Under **Files**, it can point at data in any format.

Deletes are the trap. Microsoft Learn states the rule plainly:

- Deleting the **shortcut itself** deletes only the shortcut. The target data stays.
- Deleting a **file or folder inside** the shortcut deletes it **in the target**, if your identity has write permission there.

So a clean-up notebook that runs `rm -r` on `Files/partner_feed/2025/` removes the partner's 2025 files from their storage account, not a local copy.

Other points from Learn (checked September 2026):

- Moving, renaming or deleting the target breaks the shortcut. Broken shortcuts appear in the Unidentified area.
- For external shortcuts you can turn on **caching**. OneLake keeps files it has read for 1 to 28 days, which cuts cross-cloud egress costs.

- With Direct Lake over SQL or T-SQL in delegated identity mode, the calling user's identity is not passed through to the target. Check which identity reaches the source before relying on its permissions.

Why it matters

A lakehouse held a shortcut `Files/finance_share` to a folder in the finance team's storage account. An analyst ran a notebook to free up space: it deleted every subfolder older than two years under `Files/`. Their identity had write access to the finance account for another project. Seven years of finance exports disappeared from the source. The storage account had no soft delete, and the files were gone.

Worked example: weak code

```

1 for path in notebookutils.fs.ls("Files/"):
2     if path.name.startswith("2023"):
3         notebookutils.fs.rm(path.path, True)

```

1. **Line 1.** Walks every folder under Files, shortcuts included.
2. **Line 3.** Recursive delete inside a shortcut deletes the target data, if the identity can write there.

Worked example: strong code

```

1 OWNED_PREFIXES = ("Files/landing/", "Files/tmp/")
2
3
4 def safe_to_delete(path: str, shortcut_roots: set[str]) -> bool:
5     """True only for paths this lakehouse owns, never under a shortcut."""
6     if any(path == root or path.startswith(root.rstrip("/") + "/") for
7           root in shortcut_roots):
8         return False
9     return path.startswith(OWNED_PREFIXES)

```

1. **Line 1.** An allowlist of folders this lakehouse owns.
2. **Line 6.** Anything under a shortcut is refused, whatever its name.

Shortcuts save copies. Treat every path under a shortcut as someone else's data.

Recap card

- **What a shortcut is** A folder that points to data in OneLake or external storage, without copying it.
- **Delete the shortcut vs delete inside it** Deleting the shortcut keeps the data. Deleting files inside it deletes them in the target, if you can write there.

- **Shortcut as a table** Create it under Tables, pointing at Delta data.
- **External shortcut caching** OneLake caches read files for 1 to 28 days to cut egress.

Practice

Exercise 1. Multiple choice (level 1)

`Files/partner_feed` is a shortcut to an ADLS Gen2 folder. You delete the shortcut `Files/partner_feed` itself. What happens to the data in ADLS?

- a) It is deleted too.
- b) It stays. Deleting a shortcut deletes only the shortcut object.
- c) It is moved to the lakehouse recycle bin.
- d) It is copied into OneLake before the shortcut goes.

Why?

- r1) Shortcuts do not cascade deletes. The shortcut is an independent object.
- r2) ADLS data can never be deleted from Fabric.

Exercise 2. Multiple choice (level 2)

Your identity has write access to the target storage. You delete `Files/partner_feed/2023/` where `partner_feed` is a shortcut. What happens?

- a) Only the lakehouse view of 2023 is hidden.
- b) The 2023 folder is deleted in the target storage account.
- c) The delete fails: shortcuts are read-only.
- d) The shortcut breaks, but the data stays.

Why?

- r1) Deleting a file or folder within a shortcut deletes it in the target when you have permission there.
- r2) Fabric syncs deletes back every night.

Exercise 3. Fill the gap (level 2)

Complete the rule for where to create a shortcut so the SQL analytics endpoint sees it as a table.

1 Create the shortcut under ____ (1), pointing at data in ____ (2) format.

Exercise 4. Write the function (level 3)

Write `safe_to_delete(path: str, shortcut_roots: set[str]) -> bool` for a clean-up job. Return `False` if `path` is a shortcut root or lies under one. Otherwise return `True` only if it starts with `"Files/landing/"` or `"Files/tmp/"`. Treat `"Files/partner"` and `"Files/partner_old/x"` as different folders.

Starter code:

```
1 def safe_to_delete(path, shortcut_roots):
2     return path.startswith("Files/")
```

Exercise 5. Architecture design (level 4)

Marketing's campaign data lives in an S3 bucket, updated daily. Your lakehouse needs it for Spark and SQL. Choose the set-up.

Options: An S3 shortcut, A nightly copy pipeline into Files.

1. How the data reaches the lakehouse
2. Where the shortcut sits, given the data is Delta
3. Cross-cloud reads
4. Identity used by the connection

Sources

- Shortcut targets, Tables vs Files behaviour, no cascading deletes, deletes inside a shortcut reach the target with permission, caching 1 to 28 days, delegated identity not passed through. (verified: <https://learn.microsoft.com/en-us/fabric/onelake/onelake-shortcuts> (updated 2026-07-13))
- `notebookutils.fs.ls(path)` lists a directory and `notebookutils.fs.rm(path, recurse)` deletes; `MSSparkUtils` was renamed `NotebookUtils`. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-file-system> and [../notebook-utilities](#))

C3-L2 Mirroring operational databases into OneLake

Concept

Mirroring brings an external database into Fabric as a unit. For sources that store data in their own formats, such as Azure SQL Database, Azure SQL Managed Instance, Azure Cosmos DB, Snowflake or Google BigQuery, Fabric **continuously replicates** the chosen tables into OneLake as Delta tables, in

near real time. You get a mirrored database item and a **SQL analytics endpoint** over it, read-only, like a lakehouse's.

Shortcut or mirroring? Microsoft Learn's rule:

- **Shortcut** when you want specific data, already in an open format, in your OneLake view without copying it.
- **Mirroring** when you want an external database or catalog, and the data behind it, in Fabric as a unit. It replicates when the source uses a proprietary format.

Cost. For database mirroring and open mirroring, the background compute that replicates data does not consume capacity. Mirrored storage is free up to **1 TB per capacity unit**: an F64 includes 64 TB of mirroring storage. Queries on the mirrored data (Spark, SQL, Power BI) use capacity as usual.

Limits come per source. For Azure SQL Database, checked September 2026:

- at most 1,000 mirrored tables per database,
- tables whose primary key uses `datetime2(7)`, `datetimeoffset(7)` or `time(7)` cannot be mirrored,
- since April 2025 tables without a primary key can be mirrored, but existing ones are not added automatically, even with "Automatically mirror future tables" turned on.

Status varies too. Most sources are generally available. Some, such as Azure Database for MySQL, Dremio and SharePoint List, were in preview in September 2026. Check the mirroring overview page before you build on a source.

After you set up a mirror, compare row counts per table between source and mirror (lesson A6-L3). The limits above fail quietly: an ineligible table is simply not there.

Why it matters

A team mirrored an Azure SQL order database and built a dashboard on it. Weeks later, finance noticed that returns were missing. The returns table's primary key included a `datetime2(7)` column, so it had never been mirrored. Nothing errored. A per-table row-count check between source and mirror on the first day would have listed it as absent.

Worked example: weak code

```
❶ -- Mirror set up for all tables. Dashboard built on the mirror.
```

```
2 SELECT COUNT(*) FROM mirror_orders.dbo.orders;
```

1. **Line 1.** Assumes every table was eligible and replicated.

Worked example: strong code

```
1 -- Run in the source database: tables and row counts
1 SELECT s.name AS schema_name, t.name AS table_name, SUM(p.rows) AS
   source_rows
3 FROM sys.tables AS t
4 JOIN sys.schemas AS s ON s.schema_id = t.schema_id
5 JOIN sys.partitions AS p ON p.object_id = t.object_id AND p.index_id IN
   (0, 1)
6 GROUP BY s.name, t.name;
7
8 -- Run on the mirror's SQL analytics endpoint: which tables arrived
9 SELECT TABLE_SCHEMA, TABLE_NAME
10 FROM INFORMATION_SCHEMA.TABLES
11 WHERE TABLE_TYPE = 'BASE TABLE';
```

1. **Line 2.** The expected list and sizes, from the source.

2. **Line 10.** The actual list in the mirror. Any table missing here was not mirrored.

Mirroring replicates operational databases for free compute and generous storage, within per-source limits. Reconcile the table list after setup.

Recap card

- **Shortcut or mirroring?** Shortcut for open-format data in storage. Mirroring for databases and catalogs, replicated when proprietary.
- **Mirroring cost** Replication compute is free. Storage is free up to 1 TB per CU. Queries use capacity.
- **Azure SQL Database mirroring limits** Up to 1,000 tables. Primary keys on datetime2(7), datetimeoffset(7) or time(7) block a table.
- **After setting up a mirror** Reconcile table lists and row counts with the source.

Practice

Exercise 1. Multiple choice (level 1)

You need an Azure SQL Database's tables in Fabric, kept up to date, for reporting. Shortcut or mirroring?

- a) Shortcut: shortcuts connect to Azure SQL Database.
- b) Mirroring: it replicates the database into OneLake continuously.
- c) Neither: export CSV files nightly.
- d) A shortcut under Tables.

Why?

- r1) Azure SQL stores data in a proprietary format, so Fabric must replicate it, which is what mirroring does.
- r2) Mirroring is always cheaper than shortcuts.

Exercise 2. Predict the output (level 2)

Using the documented rule of 1 TB of free mirroring storage per capacity unit, what does this print?

```
1 capacity_units = 32
2 mirrored_tb = 45
3 free_tb = capacity_units * 1
4 print(free_tb, max(0, mirrored_tb - free_tb))
```

Exercise 3. Fill the gap (level 2)

List the base tables that arrived in the mirror, on its SQL analytics endpoint.

```
1 SELECT TABLE_SCHEMA, TABLE_NAME
2 FROM _____.TABLES
3 WHERE TABLE_TYPE = '_____'
```

Exercise 4. Write the function (level 3)

Write

```
missing_from_mirror(source: dict[str, int], mirror: dict[str, int]) ->
```

list[str] for a reconciliation job. Both map `schema.table` to a row count.

Return, sorted, the tables that are absent from the mirror, or whose mirror count differs from the source count.

Starter code:

```
1 def missing_from_mirror(source, mirror):
2     return [t for t in source if t not in mirror]
```

Exercise 5. Architecture design (level 4)

Plan mirroring for an Azure SQL order database with 1,400 tables. Some primary keys use `datetime2(7)`.

Options: The tables reporting needs, under the 1,000-table limit, All 1,400.

1. Which tables to mirror
2. Tables with `datetime2(7)` primary keys
3. After setup

Sources

- Shortcuts vs mirroring guidance, and mirroring replicates when the source stores data in a proprietary format. (verified: <https://learn.microsoft.com/en-us/fabric/onelake/unify-data> (updated 2026-07-15))
- Replication compute is free and does not consume capacity; mirroring storage is free at 1 TB per CU; MySQL, Dremio and SharePoint List sources are in preview; BigQuery is GA. (verified: <https://learn.microsoft.com/en-us/fabric/mirroring/overview> (updated 2026-08-28))
- Azure SQL Database mirroring: 1,000 tables maximum, datetime2(7)/datetimeoffset(7)/time(7) primary keys unsupported, tables without primary keys supported since April 2025 but not added automatically. (verified: <https://learn.microsoft.com/en-us/fabric/mirroring/azure-sql-database-limitations> (updated 2026-02-26))

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric

Concept

Your Track A pipelines write gold tables into Unity Catalog. **Mirroring Azure Databricks Unity Catalog** makes them readable in Fabric without copying them. Unlike database mirroring, **no data moves**. Fabric mirrors only the catalog structure (catalogs, schemas, tables) and reads the underlying Delta files through shortcuts.

You get a mirrored Azure Databricks item with a SQL analytics endpoint, and you can build Power BI reports on it in Direct Lake mode. By default, **Automatically sync future catalog changes for the selected schema** is on, so new tables appear. Propagation takes from seconds to several minutes.

Three rules decide whether it works and whether it is safe (Microsoft Learn, checked September 2026):

1. Some table types do not mirror: tables with row filters or column masks (RLS/CLM policies), Lakehouse federated tables, Delta Sharing tables, streaming tables, views and materialized views. The ADLS Gen2 account behind the Databricks workspace must also be reachable from Fabric.

2. Unity Catalog permissions do not carry over. Grants on catalogs, schemas and tables in Databricks do not apply in Fabric. You rebuild access

with Fabric's model: workspace roles, item permissions, SQL permissions, OneLake security.

3. One credential reads everything. The credential used to create the connection is used for all data queries. Unity Catalog policies such as row filters, column masks or ABAC are not enforced at the storage layer, so they would not apply to direct storage access anyway.

Put together: mirror tables that are safe to share at the level you grant in Fabric. For data that needs row-level rules, publish a gold table without the policy that already contains only what the audience may see, or apply the rules again in Fabric.

Why it matters

A team mirrored their gold schema so that regional managers could build Power BI reports. In Databricks, a row filter limited each manager to their region (lesson A4-L3). The filtered table did not appear in Fabric at all, so the team created an unfiltered copy for the mirror, with a connection that could read every row. Every manager with Viewer access now saw every region. The rules had lived in Unity Catalog, and nothing had been rebuilt in Fabric.

Worked example: weak code

- ❶ Mirror: catalog gold_prod, schema sales, all tables, auto-sync on
- ❷ Connection: service principal with SELECT on the whole catalog
- 3 Fabric workspace: regional managers as Viewers
- ❸ Assumption: Unity Catalog row filters still apply

1. **Line 1.** Tables with row filters are not mirrored, and any new table appears automatically.
2. **Line 2.** This credential reads every mirrored table for every user.
3. **Line 4.** Unity Catalog permissions and policies do not carry over to Fabric.

Worked example: strong code

- ❶ Mirror: catalog gold_prod, schema sales_shareable (tables approved for all managers)
- ❷ Connection: service principal with SELECT on sales_shareable only
- ❸ Regional detail: rebuilt in Fabric with row-level security on the semantic model
- 4 Check: row counts per mirrored table match Databricks after each release

1. **Line 1.** A schema of tables that are safe for the whole Fabric audience.
2. **Line 2.** Least privilege on the one credential that reads everything.
3. **Line 3.** Access rules recreated where the users are.

Mirroring Unity Catalog copies no data and no permissions. Decide what is shareable, scope the connection, and rebuild access rules in Fabric.

Recap card

- **What mirroring Unity Catalog moves** Only catalog metadata. Data is read in place through shortcuts.
- **Unity Catalog tables that do not mirror** Tables with row filters or column masks, federated, Delta Sharing and streaming tables, views and materialized views.
- **Unity Catalog permissions in Fabric** Not carried over. Rebuild access with Fabric's permission model.
- **Which identity reads mirrored data?** The connection's credential, for all queries. Scope it tightly.

Practice

Exercise 1. Multiple choice (level 1)

When Fabric mirrors an Azure Databricks Unity Catalog, what moves into OneLake?

- a) A full copy of every table's data
- b) Only the catalog structure. Data is read through shortcuts.
- c) Nightly snapshots of changed partitions
- d) Nothing. Fabric queries Databricks SQL warehouses.

Why?

- r1) Unity Catalog tables are already open Delta files, so Fabric mirrors metadata and reads the files in place.
- r2) Copying is blocked by Databricks licensing.

Exercise 2. Multiple choice (level 2)

A gold table in Unity Catalog has a row filter. You select its schema for mirroring. What happens to that table?

- a) It is mirrored with the row filter enforced in Fabric.
- b) It is mirrored without the filter.
- c) It is not mirrored: tables with RLS or CLM policies are unsupported.
- d) It is mirrored as a view.

Why?

- r1) The limitations page lists tables with RLS/CLM policies as unsupported.

- r2) Row filters make tables read-only.

Exercise 3. Put in order (level 2)

Order the steps to publish Databricks gold tables to Fabric users safely.

- Create a connection identity with SELECT on that schema only
- Compare table lists and row counts with Databricks
- Create the mirrored Azure Databricks item for that schema
- Choose tables safe for the whole Fabric audience, in a dedicated schema
- Grant Fabric access with workspace roles, SQL permissions or model RLS

Exercise 4. Write the function (level 3)

Write `mirror_eligible(tables: list[dict[str, object]]) -> list[str]`. Each dict has `name` (str), `table_type` (str) and `has_row_filter_or_mask` (bool). Return, sorted, the names that can be mirrored: `table_type` must be `"MANAGED"` or `"EXTERNAL"`, and the table must not have a row filter or column mask. Streaming tables, views, materialized views, federated and Delta Sharing tables arrive with other `table_type` values.

Starter code:

```
1 def mirror_eligible(tables):  
2     return [t["name"] for t in tables]
```

Exercise 5. Architecture design (level 4)

Regional managers must each see only their region's sales in Power BI. The data is a Unity Catalog gold table with a row filter. Choose the design.

Options: An unfiltered copy of the table, for all managers, An unfiltered table, with row-level security rebuilt in the Fabric semantic model.

1. What Fabric mirrors
2. Connection identity
3. Manager access to the mirror's SQL endpoint

Sources

- Only the Azure Databricks catalog structure is mirrored and data is accessed through shortcuts, with no data movement; auto-sync of future catalog changes is on by default; propagation takes seconds to several minutes. (verified: <https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks> (updated 2025-10-24))
- Unsupported table types: RLS/CLM tables, Lakehouse federated tables, Delta Sharing tables, streaming tables, views and materialized views; the

Databricks ADLS account must be accessible to Fabric. (verified: <https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks-limitations> (updated 2025-12-19))

- Unity Catalog permissions are not mirrored; the connection credential is used for all data queries; UC policies are not enforced at the storage layer. (verified: <https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks-security> (updated 2026-05-01))
- Unity Catalog `information_schema.tables` reports `table_type` values including `MANAGED`, `EXTERNAL`, `VIEW`, `MATERIALIZED_VIEW`, `STREAMING_TABLE` and `FOREIGN`. (verified: <https://docs.databricks.com/aws/en/sql/language-manual/information-schema/tables>)

C4 Ingestion and orchestration

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Data Factory pipelines, Dataflows Gen2 and copy jobs.

C4-L1 Choosing how data gets into Fabric

Concept

Fabric offers several ways to move data. They overlap, and picking by habit creates pipelines nobody needs. Microsoft Learn's data movement guide (updated April 2026) gives a starting rule by layer:

- **Curated data from an existing system, for reporting: Mirroring.** If the source already does its own ETL and you need the result in Fabric, mirroring is the simplest and cheapest choice (lesson C3-L2).
- **Raw ingestion into bronze: Copy job.** It handles bulk copy, incremental copy and change data capture (CDC), with scheduling and state tracking built in, without building a pipeline.
- **Low-latency events: Eventstream** (lesson C7-L1).

Then add tools only where they earn their place:

- **Pipeline** (Data Factory): orchestration. Run activities in order, branch on success or failure, loop, retry, and call notebooks, stored procedures, dataflows and Copy jobs. A pipeline with a copy activity also moves data, but you manage incremental state yourself.
- **Dataflow Gen2:** Power Query, low-code transformation for analysts, with a data destination per query.
- **Spark notebook:** code-first transformation at scale, the closest match to your Databricks work (lesson C5-L1).

A simple way to decide:

1. Does the source need to be replicated as a whole database? Mirroring.
2. Is it a copy of tables or files into bronze? Copy job.
3. Are there events that must arrive within seconds? Eventstream.
4. Is there transformation? Spark for engineers, Dataflow Gen2 for low-code authors.
5. Do several steps need ordering, branching and alerts? Wrap them in a pipeline.

Two warnings. A pipeline that only copies tables re-creates what a Copy job does, with more to maintain. And Dataflow Gen2 is not a replacement for Spark on large volumes. Check your capacity usage (lesson C10-L3) before you standardise on one tool.

Why it matters

A team built 40 pipelines, each with a copy activity, a lookup of the last loaded timestamp from a control table, and a stored procedure to update it. Two of them had an off-by-one error in the watermark and silently skipped rows every night. A Copy job per source would have tracked incremental state itself, with nothing for the team to get wrong.

Worked example: weak code

- ❶ Source: Azure SQL CRM (200 tables) -> 200 pipelines, copy activity + control table
- ❷ Source: Clickstream events -> hourly copy activity from storage
- ❸ Transform: Dataflow Gen2 on 400 M rows per day

1. **Line 1.** Hand-built incremental state for a whole database: mirroring or a Copy job does this for you.
2. **Line 2.** Events that need low latency, handled as hourly batches.
3. **Line 3.** A low-code tool on engineering-scale volume. Spark suits this better.

Worked example: strong code

- ❶ Source: Azure SQL CRM (200 tables) -> Mirroring (curated reporting copy)
- ❷ Source: Supplier files in ADLS -> Copy job, incremental, into bronze
- 3 Source: Clickstream events -> Eventstream into an eventhouse
- 4 Transform: Spark notebooks, bronze to gold
- ❸ Orchestration: one pipeline runs the notebooks in order, retries, alerts on failure

1. **Line 1.** Free replication compute, no incremental logic to maintain.
2. **Line 2.** State tracking built in.
3. **Line 5.** The pipeline orchestrates. It does not re-create copying.

Mirroring for curated databases, Copy job for bronze, Eventstream for events, Spark or Dataflow Gen2 for transformation, and pipelines to orchestrate.

Recap card

- **Microsoft's starting rule by layer** Gold from existing systems:
Mirroring. Bronze ingestion: Copy job. Low-latency events: Eventstream.

- **What a pipeline is for** Orchestration: ordering, branching, loops, retries and alerts across activities.
- **Spark or Dataflow Gen2?** Spark for code-first, large-volume engineering. Dataflow Gen2 for low-code Power Query authors.
- **Copy activity vs Copy job for incremental loads** Copy activity needs hand-built state. Copy job tracks the last successful run itself.

Practice

Exercise 1. Multiple choice (level 1)

You need raw CSV deliveries from ADLS copied into a bronze lakehouse every hour, only new files each time. Which tool does Microsoft Learn recommend starting with?

- a) Mirroring
- b) Copy job
- c) Dataflow Gen2
- d) A pipeline with a copy activity and a control table

Why?

- r1) Copy job supports incremental copy and scheduling natively for bronze ingestion.
- r2) Copy job is the only tool that reads ADLS.

Exercise 2. Architecture design (level 2)

Pick the tool for each need.

Options: Mirroring, Copy job, Eventstream, Spark notebook, Pipeline.

1. An ERP database's curated tables, kept current for reporting
2. Sensor events that must trigger alerts within seconds
3. Joining and cleaning 300 M rows a day into silver
4. Run copy, then three notebooks in order, and email on failure

Exercise 3. Put in order (level 2)

Order the questions in the decision sequence from this lesson.

- Transformation needed, and by whom?
- Several steps that need ordering and alerts?
- Copy tables or files into bronze?
- Replicate a whole database or catalog?
- Events that must arrive within seconds?

Exercise 4. Write the function (level 3)

Write `recommend(need: str) -> str` encoding the decision rule. Accepted needs and answers:

- `"replicate_database"` → `"Mirroring"`
- `"bronze_copy"` → `"Copy job"`
- `"low_latency_events"` → `"Eventstream"`
- `"code_transform"` → `"Spark notebook"`
- `"lowcode_transform"` → `"Dataflow Gen2"`
- `"orchestrate"` → `"Pipeline"`

Raise `ValueError` for anything else, with the unknown need in the message.

Starter code:

```
1 def recommend(need):  
2     return "Pipeline"
```

Exercise 5. Multiple choice (level 4)

A team copies 60 tables from Azure SQL into bronze with 60 pipelines, each using a copy activity, a control table for the last watermark, and a stored procedure to update it. Two tables lose rows every night. What change removes the whole class of bug?

- a) Add a retry to each copy activity.
- b) Move the tables to Copy jobs with incremental copy, which track state themselves.
- c) Run the pipelines twice a night.
- d) Replace the stored procedure with a Dataflow Gen2.

Why?

- r1) Copy job manages the state of the last successful run, so there is no hand-written watermark logic to get wrong.
- r2) Pipelines cannot copy incrementally.

Sources

- Data movement guide: Mirroring for gold from existing systems, Copy job for bronze ingestion, Eventstream for low-latency events; copy activity requires managing incremental state yourself. (verified: <https://learn.microsoft.com/en-us/fabric/data-factory/decision-guide-data-movement> (updated 2026-04-13))

- Copy job supports full copy, watermark-based and CDC-based incremental copy, and tracks the state of the last successful run. (verified: <https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job> (updated 2026-09-17))

C4-L2 Incremental copy with a watermark, or with CDC

Concept

A **Copy job** copies data in one of two modes. **Full copy** moves everything on every run. **Incremental copy** moves everything on the first run, then only new or changed data. The job tracks the state of the last successful run itself (Microsoft Learn, checked September 2026).

Incremental copy works in one of two ways:

- **Watermark.** You pick an incremental column, such as a `ROWVERSION`, a datetime, a date or an increasing integer. Each subsequent load copies rows whose value is **larger than any value copied before**.
- **CDC.** If change data capture is enabled on the source database, the job reads its change stream and copies every insert, update and delete since the last run. You do not choose a column.

The watermark method has two blind spots, and both follow from "larger than":

- It **cannot see deletes**. A deleted row simply stops appearing, and the destination keeps it.
- It **misses late rows**. A row committed after the run, with a watermark value at or below the one already copied, is never picked up. A `modified_at` set by the application clock, or several rows written in the same second, causes this.

Prefer CDC when you need deletes, or a `ROWVERSION` column, which the database increments on every change. Microsoft also lets you reset incremental copy to a full copy, for the whole job or per table, to recover.

Choose the **update method** for the destination on purpose. The default is **Append**, which keeps full history, and appends every changed row again as a new row. **Merge** updates existing rows by key (the primary key by default), **Overwrite** replaces the data, and **SCD Type 2** keeps versioned history. For a silver table that must hold one row per key, choose Merge.

Why it matters

An order table was copied incrementally on `modified_at`, set by the ordering app. At 23:59:59 one batch committed after the nightly run had read up to 23:59:59. Those rows carried the same timestamp as the last copied row, so they were never picked up. Finance found 212 missing orders at month end. CDC, or a `ROWVERSION` column, would have caught them.

Worked example: weak code

```
1 def next_batch(rows, last_wm):
  ❶ batch = [r for r in rows if r["modified_at"] >= last_wm]
  ❷ return batch, max(r["modified_at"] for r in batch)
```

1. Line 2.

= re-copies the last run's boundary rows every time, which Append turns into duplicates.

2. Line 3. Fails with ValueError on a run with no new rows.

Worked example: strong code

```
1 Row = dict[str, int]
2
3
4 def next_batch(rows: list[Row], last_wm: int | None) -> tuple[list[Row],
  int | None]:
5     """Rows with a watermark above last_wm, and the new watermark."""
  ❶ batch = [r for r in rows if last_wm is None or r["rv"] > last_wm]
7     if not batch:
  ❷ return [], last_wm
9     return batch, max(r["rv"] for r in batch)
```

1. Line 6. Strictly greater, as Copy job does, on a ROWVERSION-style column.

2. Line 8. An empty run keeps the old watermark instead of failing.

Watermarks copy rows above the last value seen: no deletes, no late rows. Use CDC or a `ROWVERSION` column, and Merge for one row per key.

Recap card

- **Copy job incremental copy** First run copies everything. Later runs copy new or changed data. State is tracked by the job.
- **Watermark blind spots** Cannot see deletes. Misses late rows at or below the stored watermark.
- **When to use CDC** When deletes must replicate, or changes arrive out of order.

- **Copy job update methods** Append (default), Merge, Overwrite, SCD Type 2. Merge for one row per key.

Practice

Exercise 1. Multiple choice (level 1)

A Copy job uses watermark-based incremental copy on `modified_at`. A row is deleted in the source. What happens in the destination?

- a) The row is deleted on the next run.
- b) Nothing. Watermark-based incremental copy cannot detect deletes.
- c) The row is marked with a soft-delete flag.
- d) The job fails until you reset it.

Why?

- r1) A watermark only finds rows with a larger value. A deleted row has no value to find.
- r2) Deletes are never supported by Copy job.

Exercise 2. Predict the output (level 2)

Run 1 copied up to watermark 105. A late row with `rv` 104 commits afterwards. What does run 2 copy?

```
1 rows = [{"id": 1, "rv": 101}, {"id": 2, "rv": 105}, {"id": 3, "rv": 104}, {"id": 4, "rv": 107}]
2 last_wm = 105
3 batch = [r["id"] for r in rows if r["rv"] > last_wm]
4 print(batch)
```

Exercise 3. Fill the gap (level 2)

Complete the Copy job settings for a silver table that must hold one current row per `order_id` and reflect source deletes.

```
1 Incremental method: ____ (1)
2 Update method: ____ (2)
```

Exercise 4. Write the function (level 3)

Write `next_batch(rows, last_wm)` for a hand-built incremental load that behaves like Copy job's watermark method. `rows` is a list of dicts with an integer `rv`. `last_wm` is `None` before the first run. Return `(batch, new_wm)`: rows with `rv` strictly greater than `last_wm` (all rows when `last_wm` is `None`), and the largest `rv` in the batch. On an empty batch return `([], last_wm)`.

Starter code:

```

1 def next_batch(rows, last_wm):
2     batch = [r for r in rows if r["rv"] >= last_wm]
3     return batch, max(r["rv"] for r in batch)

```

Exercise 5. Architecture design (level 4)

Configure incremental copy of an orders table into silver. Orders are updated and sometimes deleted. The source has CDC enabled.

Options: CDC, Watermark on modified_at.

1. Incremental method
2. Update method
3. After an outage caused missed changes

Sources

- Copy job modes, watermark column types, strictly larger watermark rule, CDC captures deletes, watermark cannot detect deletes, reset to full copy per job or table. (verified: <https://learn.microsoft.com/en-us/fabric/data-factory/incremental-copy-job> (updated 2026-09-15) and [.../what-is-copy-job](https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job) (updated 2026-09-17))
- Update methods Append (default), Merge (primary key by default), Overwrite and SCD Type 2. (verified: <https://learn.microsoft.com/en-us/fabric/data-factory/what-is-copy-job> (updated 2026-09-17))

C4-L3 Pipelines that fail fast, retry sensibly and alert

Concept

A Data Factory **pipeline** runs activities: Copy job, notebook, stored procedure, Dataflow, If Condition, ForEach, Until, and more. By default a pipeline holds up to 120 activities. Three settings on each activity decide how a failure behaves (Microsoft Learn, checked September 2026):

- **Timeout.** How long an activity may run before it is stopped. The default is **12 hours**, and the maximum is seven days, written `D.HH:MM:SS`. A notebook that normally takes 10 minutes and hangs will hold its slot, and its capacity, for 12 hours unless you lower this.
- **Retry.** Retries happen only if you enable them. The retry count then defaults to 1, with **30 seconds** between attempts. Retry steps that fail for passing reasons, such as a busy source or a network drop. Do not retry a step that failed because its data is wrong. It will fail again, later.

- **Dependency branches.** Each activity connects to the next through a condition: **Upon Success**, **Upon Failure**, **Upon Completion** (either way) or **Upon Skip**. Put alerts and clean-up on Upon Failure. Upon Completion runs even when the step succeeded.

Parameters and expressions keep one pipeline reusable across sources and environments. Expressions start with @: `@pipeline().parameters.wave`. Inside a string, `@{...}` interpolates: `"landing/{pipeline().parameters.wave}"`. The result of interpolation is always a **string**, so `"@{pipeline().parameters.max_rows}"` passes text, not a number. When a setting needs a number, use the bare expression `@pipeline().parameters.max_rows`.

Keep business logic out of the pipeline. Transformations belong in notebooks and SQL, where they can be tested (lesson A9-L1). The pipeline decides order, retries and alerts.

Why it matters

A nightly pipeline ran a 10-minute notebook that sometimes hung on a locked source. With the default 12-hour timeout and no failure branch, each hang held its capacity until mid-morning. Downstream reports showed yesterday's data, and nobody was told. A 30-minute timeout, one retry and an Upon Failure email would have raised the alarm at 02:30 instead of 10:00.

Worked example: weak code

```
1 Activity: nb_silver_orders (notebook)
  1 Timeout: default
  3 Retries: not enabled
  2 Activity: nb_gold_sales depends on nb_silver_orders: Upon Completion
  3 Activity: send_failure_email depends on nb_gold_sales: Upon Completion
```

1. **Line 2.** 12 hours for a 10-minute job: a hang goes unnoticed until morning.
2. **Line 4.** Gold runs even when silver failed, publishing stale or partial data.
3. **Line 5.** The 'failure' email is sent every night, so people learn to ignore it.

Worked example: strong code

```
1 Activity: nb_silver_orders (notebook)
  1 Timeout: 0.00:30:00
  3 Retries: enabled, 1 retry, 60 seconds apart
  2 Activity: nb_gold_sales depends on nb_silver_orders: Upon Success
  3 Activity: send_failure_email depends on nb_silver_orders: Upon Failure
  6 Activity: send_failure_email_gold depends on nb_gold_sales: Upon Failure
```

1. **Line 2.** Three times the normal run time: a hang fails within half an hour.

- 2. **Line 4.** Gold runs only on good silver.
- 3. **Line 5.** Alerts fire only when something failed.

Set timeouts near the real run time, retry only transient failures, chain on Upon Success and alert on Upon Failure.

Recap card

- **Activity timeout default** 12 hours, maximum seven days, format D.HH:MM:SS. Set it near the real run time.
- **Activity retries** Only when enabled. Count defaults to 1, interval 30 seconds. Retry transient failures only.
- **Dependency branches** Upon Success, Upon Failure, Upon Completion (either way), Upon Skip.
- **@{...} vs @... @{...}** inside text always yields a string. A bare @ expression keeps the value's type.

Practice

Exercise 1. Multiple choice (level 1)

You leave an activity's timeout at its default. How long can it run before Fabric stops it?

- a) 30 minutes
- b) 1 hour
- c) 12 hours
- d) 7 days

Why?

- r1) The documented default timeout is 12 hours. Seven days is the maximum you can set.
- r2) Timeouts match the capacity's pause schedule.

Exercise 2. Multiple choice (level 2)

An alert activity is attached to a notebook with **Upon Completion**. When does it run?

- a) Only when the notebook fails
- b) Only when the notebook succeeds
- c) Whether the notebook succeeds or fails
- d) Only when the notebook is skipped

Why?

- r1) Upon Completion runs after success or failure. Upon Failure is the failure-only branch.
- r2) Completion means the capacity finished all jobs.

Exercise 3. Fill the gap (level 2)

Build a folder path from the pipeline parameter `wave`, and pass the numeric parameter `max_rows` as a number.

```
1 Folder path: landing/____(1)pipeline().parameters.wave}/
2 Max rows: ____ (2)pipeline().parameters.max_rows
```

Exercise 4. Write the function (level 3)

Write `parse_timeout(value: str) -> int` for a pipeline linter. It parses the D.HH:MM:SS format and returns the total seconds. Raise `ValueError` if the format is wrong, if hours are above 23 or minutes or seconds above 59, or if the total exceeds 7 days.

Starter code:

```
1 def parse_timeout(value):
2     days, rest = value.split(".")
3     h, m, s = rest.split(":")
4     return int(days) * 86400 + int(h) * 3600 + int(m) * 60 + int(s)
```

Exercise 5. Spot the bug (level 4)

This pipeline publishes gold tables even when silver fails. Tap the line that causes it, then rewrite the dependencies so gold runs only on success and an alert fires on failure.

```
1 Activity: nb_silver_orders
2   Timeout: 0.00:30:00
3 Activity: nb_gold_sales depends on nb_silver_orders: Upon Completion
4 Activity: email_on_failure depends on nb_silver_orders: Upon Failure
```

Write down the line number of the bug. Then write the fix.

Sources

- Activity timeout defaults to 12 hours with a seven-day maximum in D.HH:MM:SS; retries when enabled default to 1 with a 30-second interval; up to 120 activities per pipeline; UponSuccess, UponFailure, UponCompletion and UponSkip branches. (verified: <https://learn.microsoft.com/en-us/fabric/data-factory/activity-overview> (updated 2026-08-10))

- Pipeline expressions use `@pipeline().parameters`; string interpolation with `@{...}` always returns a string. (verified: <https://learn.microsoft.com/en-us/fabric/data-factory/expression-language> (updated 2024-12-18))

C5 Spark notebooks in Fabric

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Differences from Databricks, environments, V-Order and Delta interoperability.

C5-L1 From Databricks notebooks to Fabric notebooks

Concept

Fabric notebooks run the same PySpark you wrote in Track A. The surroundings differ, and those differences break ported code (Microsoft Learn, checked September 2026).

Utilities. There is no `dbutils`. Fabric has **NotebookUtils**, formerly `MSSparkUtils`. The `mssparkutils` namespace still works but is being retired, so write `notebookutils`:

Databricks	Fabric
<code>dbutils.fs.ls(path)</code>	<code>notebookutils.fs.ls(path)</code>
<code>dbutils.secrets.get(scope, key)</code>	<code>notebookutils.credentials.getSecret("https://<vault>.vault.azure.net/", "name")</code>
<code>dbutils.notebook.run(path, timeout, args)</code>	<code>notebookutils.notebook.run(path, timeout_seconds, arguments)</code>
<code>dbutils.notebook.exit(value)</code>	<code>notebookutils.notebook.exit(value)</code>

Secrets live in **Azure Key Vault**, not in Databricks secret scopes. You need Get permission on the vault. Output that prints a secret is redacted, but do not print secrets.

Child notebooks time out at 90 seconds by default.

`notebookutils.notebook.run(path, timeout_seconds=90, ...)`. A child notebook that runs for five minutes fails unless you pass a longer timeout. For several notebooks, `notebookutils.notebook.runMultiple` runs a DAG of them in parallel.

Default lakehouse. A notebook can attach a default lakehouse. Relative paths such as `Files/landing` and bare table names resolve against it. Name the lakehouse and schema in shared code (lesson C2-L1), so a changed attachment does not change what the code reads.

Inline installs. `%pip install` affects only the current session, and inline commands are **turned off by default in pipeline runs**. Put libraries in an environment instead (lesson C5-L2).

Runtime. Runtime 2.0 (Spark 4.1, Delta Lake 4.2, Python 3.13, Java 21) is GA and recommended for production. New workspaces still default to Runtime 1.3 (Spark 3.5), whose end of support has been announced. Set the runtime deliberately. It changes the Spark version your code meets.

Why it matters

A team ported a Databricks orchestration notebook that called six child notebooks with `mssparkutils.notebook.run(path)`. In development each child took under a minute. On month-end data, the largest took four minutes. Every run failed at 90 seconds with a timeout, and the error pointed at the parent notebook. Passing `timeout_seconds` sized to the real run time fixed it.

Worked example: weak code

```
❶ password = dbutils.secrets.get("prod", "sql-password")
❷ %pip install great_expectations
❸ result = mssparkutils.notebook.run("nb_silver_orders")
❹ df = spark.read.table("orders")
```

1. **Line 1.** `dbutils` does not exist in Fabric. Secrets come from Azure Key Vault.
2. **Line 2.** Inline installs are turned off by default in pipeline runs, and give unpinned dependencies.
3. **Line 3.** Old namespace, and a 90-second default timeout for the child.
4. **Line 4.** Depends on whichever lakehouse is attached as default, and the `dbo` schema.

Worked example: strong code

```
1 VAULT_URL = "https://kv-sales-prod.vault.azure.net/"
2
❶ password = notebookutils.credentials.getSecret(VAULT_URL, "sql-password")
4 result = notebookutils.notebook.run(
❷     "nb_silver_orders", timeout_seconds=1800, arguments={"wave":
        "2026-09"}
6 )
❸ df = spark.read.table("lh_sales.silver.orders")
```

1. **Line 3.** Key Vault through `NotebookUtils`. The vault URL belongs in config.
2. **Line 5.** A timeout sized to the child's real run time, and explicit arguments.
3. **Line 7.** Lakehouse and schema named, independent of the default attachment.

Replace `dbutils` with `notebookutils`, secrets with Key Vault, inline installs with environments, and pass real timeouts to child notebooks.

Recap card

- **Fabric replacement for dbutils** notebookutils (formerly mssparkutils).
- **Secrets in Fabric notebooks**
notebookutils.credentials.getSecret(vault_url, name), from Azure Key Vault.
- **notebookutils.notebook.run default timeout** 90 seconds. Pass timeout_seconds.
- **%pip in pipeline runs** Turned off by default. Use an environment.

Practice

Exercise 1. Multiple choice (level 1)

Which call reads a secret in a Fabric notebook?

- a) `dbutils.secrets.get("prod", "sql-password")`
- b) `notebookutils.credentials.getSecret("https://kv-sales.vault.azure.net/", "sql-password")`
- c) `os.environ["SQL_PASSWORD"]` set in the workspace settings
- d) `spark.conf.get("secret.sql-password")`

Why?

- r1) Fabric reads secrets from Azure Key Vault through notebookutils.credentials.
- r2) getSecret caches the secret in the notebook.

Exercise 2. Multiple choice (level 2)

`notebookutils.notebook.run("nb_big_load")` calls a child notebook that takes four minutes. What happens?

- a) It completes after four minutes.
- b) It fails after 90 seconds, the default timeout.
- c) It fails after 12 hours.
- d) It runs in the background and returns at once.

Why?

- r1) `run()` has `timeout_seconds` with a default of 90.
- r2) Child notebooks are limited to one Spark job.

Exercise 3. Fill the gap (level 2)

Port this Databricks line to Fabric.

```
1 files = _____.fs.ls("Files/landing")
2 value = _____.notebook.run("nb_child", _____.(2)=600)
```

Exercise 4. Write the function (level 3)

Write `port_line(line: str) -> str` for a migration helper. It rewrites Databricks utility calls to Fabric:

- `dbutils.fs.` → `notebookutils.fs.`
- `dbutils.notebook.` → `notebookutils.notebook.`
- `mssparkutils.` → `notebookutils.`

If the line contains `dbutils.secrets.`, raise `ValueError` with a message that mentions Key Vault, because it needs a manual change. Leave other text unchanged.

Starter code:

```
1 def port_line(line):
2     return line.replace("dbutils", "notebookutils")
```

Exercise 5. Spot the bug (level 4)

This notebook runs from a nightly pipeline. It worked when run by hand and fails in the pipeline. Tap the line that fails there, and rewrite the notebook without it.

```
1 %pip install pandera==0.33.1
2 import pandera.pandas as pa
3 df = spark.read.table("lh_sales.silver.orders").toPandas()
4 SCHEMA.validate(df, lazy=True)
```

Write down the line number of the bug. Then write the fix.

Sources

- MsSparkUtils is renamed NotebookUtils; the mssparkutils namespace will be retired. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/notebook-utilities>)
- `notebookutils.notebook.run(path, timeout_seconds=90, arguments=None, workspace=)` returns the exit value; `runMultiple` runs a DAG. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-notebook-run>)
- `notebookutils.credentials.getSecret` reads from Azure Key Vault and needs Get permission; output redaction is attempted. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-credentials> (updated 2026-09-23))

- Inline installs are turned off by default in pipeline runs. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/library-management> (updated 2026-03-25))
- Runtime 2.0 (Spark 4.1, Delta 4.2, Python 3.13, Java 21) is GA and recommended; new workspaces default to Runtime 1.3, which is at end-of-support-announced stage. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/runtime> (updated 2026-07-24))

C5-L2 Environments, resource profiles and V-Order

Concept

An **environment** is a Fabric item that fixes what a Spark session starts with: the runtime version, compute settings, libraries and small resource files. Attach it to notebooks and Spark job definitions, or make it the **workspace default** so every notebook set to "Workspace default" inherits it. Once an environment is the workspace default, only workspace admins can change its contents.

Libraries publish in one of two modes (Microsoft Learn, checked September 2026):

- **Quick mode** installs libraries when each session starts. It is flexible, and it adds one to three minutes to start-up.
- **Full mode** builds a snapshot of the dependencies once. Microsoft recommends Full mode for pipelines, scheduled runs and shared workloads that need reproducible environments. It is the Fabric equivalent of the lock file in lesson B6-L3.

Resource profiles set a tested group of Spark and Delta settings with one property. New workspaces default to `writeHeavy`, tuned for ingestion. `readHeavyForSpark` and `readHeavyForPBI` suit read-heavy work.

V-Order is a Fabric write-time layout for Parquet files. It keeps files standard, open Parquet, and speeds up reads, especially for Power BI Direct Lake and the SQL engines. The cost is slower writes: Learn cites **around 15%** on average. Since the `writeHeavy` default, **V-Order is off by default in new workspaces** (`spark.sql.parquet.vorder.default=false`).

Turn it on where reads dominate, at the narrowest useful level:

- per table: `TBLPROPERTIES("delta.parquet.vorder.enabled" = "true")`, the best choice for gold tables that Power BI reads,
- per write: the DataFrame option `parquet.vorder.enabled`,

- per session: `spark.sql.parquet.vorder.default`, or the `readHeavyForPBI` profile, which enables it.

In Runtime 1.3 and later the older `spark.sql.parquet.vorder.enable` setting is removed. Code copied from older posts that sets it does nothing.

Why it matters

A team moved its gold layer to a new Fabric workspace and noticed Power BI reports running noticeably slower than before. The old workspace had V-Order on for every write. The new one used the `writeHeavy` default, with V-Order off. Setting `delta.parquet.vorder.enabled` on the dozen gold tables, and optimising them once, restored read performance without slowing bronze and silver ingestion.

Worked example: weak code

```
1 spark.conf.set("spark.sql.parquet.vorder.enable", "true")
2 bronze.write.mode("append").saveAsTable("lh.bronze.orders")
3 gold.write.mode("overwrite").saveAsTable("lh.gold.sales_by_region")
```

1. **Line 1.** Removed setting in Runtime 1.3 and later. It has no effect.
2. **Line 2.** Had the setting worked, bronze would pay the write cost for reads nobody makes.

Worked example: strong code

```
1 bronze.write.mode("append").saveAsTable("lh.bronze.orders")
2
3 spark.sql("""
4     CREATE TABLE IF NOT EXISTS lh.gold.sales_by_region (region STRING,
5         sales DECIMAL(19, 4))
6     USING DELTA
7     TBLPROPERTIES ("delta.parquet.vorder.enabled" = "true")
8 """)
9 gold.write.mode("overwrite").saveAsTable("lh.gold.sales_by_region")
```

1. **Line 1.** Write-heavy bronze keeps the fast default.
2. **Line 6.** V-Order only on the gold table that Power BI reads.

Pin libraries in a Full mode environment, keep the `writeHeavy` default for ingestion, and turn V-Order on for read-heavy gold tables.

Recap card

- **Environment publishing modes** Quick: installs at session start (adds minutes). Full: reproducible snapshot, recommended for pipelines.
- **V-Order in new workspaces** Off by default (`writeHeavy` profile). Write cost around 15%, faster reads.

- **Turn V-Order on for one table**

TBLPROPERTIES("delta.parquet.vorder.enabled" = "true").

- **spark.sql.parquet.vorder.enable** Removed in Runtime 1.3 and later. Use spark.sql.parquet.vorder.default or table and writer options.

Practice

Exercise 1. Multiple choice (level 1)

In a workspace created in 2026 with default settings, is V-Order applied when a notebook writes a Delta table?

- a) Yes, V-Order is always on in Fabric.
- b) No. New workspaces default to the writeHeavy profile, with V-Order off.
- c) Only for tables under Files.
- d) Only when the table is larger than 1 GB.

Why?

- r1) spark.sql.parquet.vorder.default is false in new workspaces to favour write performance.
- r2) V-Order has been retired.

Exercise 2. Multiple choice (level 2)

A nightly pipeline runs notebooks that need pandera and a pinned version of a statistics library. How should the libraries be delivered?

- a) %pip install at the top of each notebook
- b) An environment in Full mode, attached to the notebooks or set as workspace default
- c) An environment in Quick mode
- d) Upload wheel files to Files and install them at runtime

Why?

- r1) Full mode builds a reproducible snapshot, and Microsoft recommends it for pipelines and scheduled runs.
- r2) Full mode is the only mode that supports pandera.

Exercise 3. Fill the gap (level 2)

Turn on V-Order for one gold table only.

```
1 ALTER TABLE lh.gold.sales_by_region
2 SET TBLPROPERTIES ("_____(1)" = "_____(2)");
```

Exercise 4. Write the function (level 3)

Write `write_gold(df, table)` that overwrites a gold Delta table with V-Order enabled for this write, using the DataFrame writer option `parquet.vorder.enabled` set to `"true"`. Do not change the session-wide default.

Starter code:

```
1 def write_gold(df, table):
2     spark.conf.set("spark.sql.parquet.vorder.default", "true")
3     df.write.mode("overwrite").saveAsTable(table)
```

Exercise 5. Architecture design (level 4)

Set up Spark for a workspace with bronze and silver ingestion and a gold layer read by Power BI Direct Lake.

Options: One Full mode environment, set as workspace default, %pip in each notebook.

1. Libraries for scheduled notebooks
2. Session-wide resource profile
3. Gold tables

Sources

- Environments set runtime, compute, libraries and resources; a workspace default environment can be changed only by workspace admins. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/create-and-use-environment> (updated 2026-03-25))
- Quick mode installs at session start adding one to three minutes; Full mode is recommended for pipelines and scheduled runs. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/library-management> (updated 2026-03-25))
- V-Order is off by default in new workspaces (writeHeavy); write cost around 15%; table property, writer option and session default; `spark.sql.parquet.vorder.enable` removed in Runtime 1.3+. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/delta-optimization-and-v-order> (updated 2026-03-01))
- New workspaces default to writeHeavy; readHeavyForPBI enables V-Order. (verified: <https://learn.microsoft.com/en-us/fabric/data-engineering/resource-profiles-overview> (updated 2026-07-23))

C5-L3 Delta tables shared between Databricks and Fabric

Concept

Delta is an open format, so a table written by Databricks can be read by Fabric through a shortcut or Unity Catalog mirroring (lessons C3-L1 and C3-L3). Open does not mean every reader supports every **table feature**. A table's features decide which Fabric experiences can read it.

From Microsoft Learn's Delta interoperability matrix (updated May 2026):

Feature	Spark Runtime 2.0	SQL analytics endpoint	Direct Lake	Python notebooks
Deletion vectors	yes	yes	yes	no
Column mapping, name mode	yes	yes	yes	no
Column mapping, id mode	yes	no	yes	no
<code>TIMESTAMP_NTZ</code>	yes	no	no	yes
Liquid clustering	yes	yes	yes	no

Three more limits from the same page:

- **V2 checkpoints**: only Spark notebooks and Spark jobs read and write them. The lakehouse and the SQL analytics endpoint do not list such tables correctly.
- **Databricks-only features**, such as writing identity columns and Lakeflow Spark Declarative Pipelines tables, are not supported.
- Newer **Delta 4.x features**, such as collations and the variant type, work only in Spark notebooks and jobs, not in the SQL endpoint or Direct Lake.

Fabric's own writers differ too. Runtime 2.0 writes new tables at reader version 3 and writer version 7 with **deletion vectors** on by default. Runtime 1.3 writes reader 1, writer 2.

The practical rule: **the table's writer decides which features the table has, and every reader must support all of them**. For gold tables that Power BI or T-SQL users read, use timestamps with a time zone (`TIMESTAMP`) instead of `TIMESTAMP_NTZ`, avoid column mapping in id mode, and check `DESCRIBE DETAIL` for the protocol versions and table features before you publish.

Why it matters

A Databricks team stored event times as `TIMESTAMP_NTZ` in a gold table and shared it with Fabric through a shortcut. Spark in Fabric read it fine. The SQL analytics endpoint and the Direct Lake report did not show the table's time columns, so the Power BI team built a workaround copy with converted columns. One type choice in the writer created a second dataset.

Worked example: weak code

```
1 CREATE TABLE gold.events (  
2     event_id STRING,  
3     event_time TIMESTAMP_NTZ  
4 )  
5 TBLPROPERTIES ("delta.columnMapping.mode" = "id");
```

1. **Line 3.** Not supported by the SQL analytics endpoint or Direct Lake.
2. **Line 5.** Column mapping in id mode is not supported by the SQL analytics endpoint.

Worked example: strong code

```
1 CREATE TABLE gold.events (  
2     event_id STRING,  
3     event_time TIMESTAMP  
4 );  
5  
6 -- Before sharing: check protocol versions and table features  
7 DESCRIBE DETAIL gold.events;
```

1. **Line 3.** A time-zone-aware timestamp that every Fabric reader supports.
2. **Line 7.** Shows `minReaderVersion`, `minWriterVersion` and table features to compare with the readers' support.

Open format, but not every feature for every reader. Design shared gold tables for the weakest reader that needs them.

Recap card

- **Delta interoperability rule** Every reader must support every feature the writer put in the table.
- **TIMESTAMP_NTZ in Fabric** Spark yes. SQL analytics endpoint and Direct Lake no.
- **Runtime 2.0 default table protocol** Reader 3, writer 7, deletion vectors on. Python notebooks cannot read deletion vectors.
- **Check a table's features** `DESCRIBE DETAIL`: `minReaderVersion`, `minWriterVersion`, table features.

Practice

Exercise 1. Multiple choice (level 1)

A Databricks gold table uses `TIMESTAMP_NTZ` columns. Which Fabric reader handles them?

- a) The SQL analytics endpoint
- b) Direct Lake semantic models
- c) Fabric Spark Runtime 2.0
- d) All of them

Why?

- r1) The interoperability matrix lists `TIMESTAMP_NTZ` as supported in Spark but not in the SQL endpoint or Direct Lake.
- r2) Only Spark can read Delta tables.

Exercise 2. Multiple choice (level 2)

You create a new Delta table with a Spark notebook on Fabric Runtime 2.0 and default settings. Which reader might fail to read it?

- a) The SQL analytics endpoint
- b) A Python notebook (non-Spark) using the default Delta reader
- c) Direct Lake
- d) Fabric Spark Runtime 2.0

Why?

- r1) Runtime 2.0 writes deletion vectors by default, and Python notebooks do not support deletion vectors.
- r2) Python notebooks cannot read OneLake.

Exercise 3. Fill the gap (level 2)

Inspect a table's protocol versions and features before sharing it.

```
1 _____.(1) DETAIL gold.events;
```

Exercise 4. Write the function (level 3)

Write `unsupported(reader: str, features: set[str]) -> list[str]` using this support table from the lesson. Return the table's features the reader does not support, sorted. Raise `KeyError` for an unknown reader.

- `spark_2_0`: supports all of `deletion_vectors`, `column_mapping_name`, `column_mapping_id`, `timestamp_ntz`, `liquid_clustering`.
- `sql_endpoint`: all except `column_mapping_id` and `timestamp_ntz`.

- `direct_lake`: all except `timestamp_ntz`.
- `python_notebook`: only `timestamp_ntz`.

Starter code:

```
1 def unsupported(reader, features):  
2     return []
```

Exercise 5. Architecture design (level 4)

A Databricks team publishes a gold table that Fabric users read through the SQL endpoint and Direct Lake. Choose the table design.

Options: `TIMESTAMP`, `TIMESTAMP_NTZ`.

1. Event time column
2. Column mapping
3. Before each release

Sources

- Feature support matrix for deletion vectors, column mapping, `TIMESTAMP_NTZ` and liquid clustering across Spark runtimes, SQL analytics endpoint, Direct Lake and Python notebooks; Runtime 2.0 writes reader 3, writer 7 with deletion vectors; V2 checkpoints, identity columns writing, Lakeflow Spark Declarative Pipelines and Delta 4.x features outside Spark are limitations. (verified: <https://learn.microsoft.com/en-us/fabric/fundamentals/delta-lake-interoperability> (updated 2026-05-08))

C6 Semantic models and Direct Lake

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Star schemas, DAX best practices and Power BI reports.

C6-L1 Star schemas for semantic models

Concept

A Power BI semantic model works best on a **star schema**: fact tables surrounded by dimension tables. Microsoft's Power BI guidance (checked September 2026) defines the parts:

- **Fact tables** store observations or events: survey responses, orders, balances. They hold dimension key columns and numeric measure columns. The dimension keys decide the fact table's **grain**, for example one row per respondent per wave per brand.
- **Dimension tables** describe business entities: respondent, brand, region, date. Each has a unique key column and descriptive columns used to filter and group. A **date dimension** belongs in almost every model.

Relationships run **one-to-many** from each dimension to the fact table, and filters flow from the dimension to the fact. Microsoft recommends that you **minimise bi-directional relationships**, which can slow queries and confuse report users with unexpected filtering. Many-to-many relationships between facts and dimensions usually mean the grain is unclear.

Build the star in the gold layer, not in the model:

- Compute derived columns such as age bands or weighted flags in Spark or SQL, in gold tables. Direct Lake on SQL analytics endpoints does not support calculated columns, and on OneLake they are in preview with restrictions. Materialised calculated columns in Import models slow refresh.
- Give every fact table a clear grain in its docstring and table comment (lesson A2-L3).
- Use surrogate or natural keys consistently, and check they are unique in each dimension (lesson C2-L2 showed warehouse keys are not enforced).

A good model is boring: one fact table per grain, conformed dimensions shared between facts, single-direction one-to-many relationships, and measures that do the arithmetic (lesson C6-L2).

Why it matters

A brand tracker model joined a respondent table and a brand-awareness table with a bi-directional many-to-many relationship, because both held respondent_id and brand. Selecting a brand in a slicer filtered respondents, which filtered back to every brand those respondents had mentioned. Awareness for small brands showed as much higher than the survey measured. A single fact table at respondent-brand grain with one-way dimensions returned the correct figures.

Worked example: weak code

```
❶ respondents (respondent_id, age, region, weight, brand_list)
❷ brand_awareness (respondent_id, brand, aware)
❸ Relationship: respondents[respondent_id] <->
    brand_awareness[respondent_id], many-to-many, both directions
❹ Calculated column: respondents[age_band] = SWITCH(TRUE(), ...)
```

1. **Line 1.** A list column mixes grains: one respondent, many brands.
2. **Line 3.** Bi-directional many-to-many: filters flow both ways and inflate results.
3. **Line 4.** Derived column in the model. Not supported in Direct Lake on SQL endpoints.

Worked example: strong code

```
❶ fact_awareness (respondent_key, brand_key, date_key, region_key, aware,
    weight) grain: respondent x brand x wave
❷ dim_respondent (respondent_key, age_band, gender)
❸ dim_brand (brand_key, brand, category)
❹ dim_date (date_key, date, wave, month)
❺ dim_region (region_key, region, nation)
❻ Relationships: each dim[key] 1 -> * fact_awareness[key], single direction
```

1. **Line 1.** One fact table with a stated grain.
2. **Line 2.** age_band computed in gold, not in the model.
3. **Line 6.** Filters flow one way, from dimensions to facts.

Facts with a stated grain, dimensions with unique keys, one-to-many single-direction relationships, and derived columns computed upstream.

Recap card

- **Star schema parts** Fact tables (keys and measures, with a grain) and dimension tables (unique key, descriptive columns).
- **Relationship guidance** One-to-many, single direction. Minimise bi-directional relationships.

- **Calculated columns and Direct Lake** Not supported on SQL endpoints, preview on OneLake. Compute derived columns in gold.
- **Grain** Set by the fact table's dimension keys. State it in the table's docstring and comment.

Practice

Exercise 1. Multiple choice (level 1)

In a star schema, what decides the grain of a fact table?

- a) The number of measure columns
- b) The dimension keys it holds, and their values
- c) The number of rows
- d) The storage mode

Why?

- r1) Dimension key columns set the dimensionality, and their values set the granularity.
- r2) Grain is chosen in Power BI settings.

Exercise 2. Multiple choice (level 2)

What does Microsoft's Power BI guidance recommend about bi-directional relationships?

- a) Use them by default so slicers work everywhere.
- b) Minimise them. They can hurt performance and confuse report users.
- c) Use them only on date tables.
- d) They are not supported in Direct Lake.

Why?

- r1) Bi-directional filters can slow queries and filter in ways users do not expect.
- r2) They are deprecated.

Exercise 3. Put in order (level 2)

Order the steps to build a model for a brand tracker.

- Build the fact table with dimension keys and measure columns
- Write DAX measures
- Build dimension tables with unique keys in gold
- Create one-to-many, single-direction relationships

- State the fact table's grain

Exercise 4. Write the function (level 3)

Write `model_warnings(relationships: list[dict[str, str]]) -> list[str]` for a model review. Each relationship has `from_table`, `to_table`, `cardinality` ("one_to_many" or "many_to_many") and `direction` ("single" or "both"). Return, in input order, a warning for each problem:

- "<from>-<to>: many-to-many" for a many-to-many relationship,
- "<from>-<to>: bi-directional" for direction both.

A relationship with both problems produces both warnings, many-to-many first.

Starter code:

```
1 def model_warnings(relationships):
2     return []
```

Exercise 5. Architecture design (level 4)

A Direct Lake model on the SQL analytics endpoint needs an age band on respondents. Choose where it is built.

Options: In the gold `dim_respondent` table, in Spark or SQL, A calculated column in the semantic model.

1. Where `age_band` is computed
2. Awareness fact table grain
3. Relationship from `dim_respondent` to the fact

Sources

- Fact and dimension definitions; dimension keys set dimensionality and their values granularity. (verified: <https://learn.microsoft.com/en-us/power-bi/guidance/star-schema>)
- Microsoft recommends minimising bi-directional relationships because of performance and confusing experiences. (verified: <https://learn.microsoft.com/en-us/power-bi/guidance/relationships-bidirectional-filtering>)
- Calculated columns: Direct Lake on SQL analytics endpoints no; Direct Lake on OneLake user-context only, preview. (verified: <https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview> (updated 2026-09-02))

C6-L2 DAX measures that state their filter context

Concept

In a semantic model, arithmetic belongs in **measures**. A measure is evaluated at query time, in the **filter context** of the visual: the slicers, rows and columns that apply to that cell. A calculated column is evaluated row by row when the model refreshes and stored, which costs memory and refresh time and cannot react to slicers.

Three habits make measures correct and readable (Microsoft's DAX guidance, checked September 2026):

- **Use DIVIDE, not /.** `DIVIDE(numerator, denominator)` returns BLANK when the denominator is zero or BLANK, instead of an error or infinity. You can pass an alternate result as a third argument. Microsoft notes it is also faster than testing for zero yourself.
- **Use variables.** `VAR ... RETURN` evaluates an expression once and names it. That improves performance, readability and debugging.
- **Change filter context only with CALCULATE,** and say why.
`CALCULATE([Weighted base], ALL(dim_brand))` removes the brand filter, which is what a share-of-total denominator needs.

A weighted share, the survey version of lesson B8-L1:

```
1 Weighted aware % =  
2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *  
   fact_awareness[weight] )  
3 VAR base_w = SUM ( fact_awareness[weight] )  
4 RETURN DIVIDE ( aware_w, base_w )
```

Put the unweighted base next to it as a separate measure, `Base (n) = DISTINCTCOUNT(fact_awareness[respondent_key])`, so report users see how many respondents stand behind each percentage.

Name measures for what they return, store them in one place (a measures table or the fact table), and set their format strings. A measure that nobody can read is a measure nobody can check.

Why it matters

A report computed awareness as `SUM(aware_weight) / SUM(weight)` in a measure. In a new market with no fieldwork yet, the base was zero, and the visual showed "Infinity" and "NaN" in cells that the board pack exported to PDF. `DIVIDE` would have returned BLANK, and the empty cells would have been left empty.

Worked example: weak code

```
1 Aware % = SUM(fact_awareness[aware_weight]) / SUM(fact_awareness[weight])
2 Brand share = [Aware %] / CALCULATE([Aware %], ALL(fact_awareness))
```

1. **Line 1.** The / operator breaks on a zero base.
2. **Line 2.** ALL on the whole fact table removes every filter, including date and region. The share compares against the wrong total.

Worked example: strong code

```
1 Weighted aware % =
2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
    fact_awareness[weight] )
3 VAR base_w = SUM ( fact_awareness[weight] )
4 RETURN DIVIDE ( aware_w, base_w )
5
6 Share of aware across brands =
7 VAR this_brand = [Weighted aware %]
8 VAR all_brands = CALCULATE ( [Weighted aware %], ALL ( dim_brand ) )
9 RETURN DIVIDE ( this_brand, all_brands )
```

1. **Line 4.** BLANK on a zero base, not an error.
2. **Line 8.** Removes only the brand filter. Date, region and wave filters still apply.

Measures, not calculated columns. DIVIDE for every ratio, variables for clarity, and CALCULATE with the narrowest filter change.

Recap card

- **Measure vs calculated column** Measures compute at query time in the visual's filter context. Calculated columns are stored at refresh.
- **DIVIDE in DAX** Returns BLANK (or an alternate result) when the denominator is zero or BLANK.
- **VAR ... RETURN** Evaluates once, names parts, improves performance, readability and debugging.
- **Share of total across brands** CALCULATE([measure], ALL(dim_brand)): remove only the brand filter.

Practice

Exercise 1. Multiple choice (level 1)

What does `DIVIDE(10, 0)` return in DAX?

- a) An error
- b) BLANK
- c) 0

- d) Infinity

Why?

- r1) DIVIDE returns BLANK when the denominator is zero or BLANK, unless an alternate result is given.
- r2) DAX has no concept of division by zero.

Exercise 2. Multiple choice (level 2)

A share-of-total measure uses `CALCULATE([Weighted aware %], ALL(fact_awareness))` as its denominator. A report filters to wave 12 and region North. What does the denominator ignore?

- a) Only the brand filter
- b) Every filter on fact_awareness, including wave and region
- c) Nothing: ALL only affects slicers
- d) Only filters from other pages

Why?

- r1) ALL on the fact table removes all its filters, including those that reach it through dimensions.
- r2) ALL always removes every filter in the model.

Exercise 3. Fill the gap (level 2)

Complete the weighted share measure.

```
1 Weighted aware % =
2 ____ (1) aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
   fact_awareness[weight] )
3 VAR base_w = SUM ( fact_awareness[weight] )
4 RETURN ____ (2) ( aware_w, base_w )
```

Exercise 4. Write the function (level 3)

Write a measure `Share of aware across brands` that divides `[Weighted aware %]` by the same measure with the brand filter removed through `ALL ( dim_brand )`. Use variables named `this_brand` and `all_brands`, and `DIVIDE`.

Starter code:

```
1 Share of aware across brands = [Weighted aware %] / CALCULATE ( [Weighted
   aware %], ALL ( fact_awareness ) )
```

Exercise 5. Architecture design (level 4)

Decide where each calculation lives in a Direct Lake model on the SQL analytics endpoint.

Options: Gold table column, DAX measure, Calculated column.

1. Respondent age band
2. Weighted awareness percentage
3. Unweighted base (number of respondents)

Sources

- DIVIDE returns BLANK for a zero or BLANK denominator unless an alternate result is passed; it outperforms explicit zero checks. (verified: <https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator>)
- Variables improve performance, reliability, readability and debugging. (verified: <https://learn.microsoft.com/en-us/dax/best-practices/dax-variables>)
- Materialised calculated columns can hurt refresh performance because values are derived at refresh time. (verified: <https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-calculated-columns> (updated 2026-08-23))

C6-L3 Direct Lake, framing and the fallback you did not plan

Concept

Direct Lake lets a semantic model read Delta tables in OneLake directly, loading only the columns a query needs into memory. It combines Import-like query speed with no copy of the data. A refresh does not reload data. It performs **framing**: it records which Delta table versions the model uses. Framing takes seconds, and afterwards queries see the data as of that frame. Automatic updates, a schedule or an API call can trigger framing.

There are two kinds (Microsoft Learn, updated September 2026):

- **Direct Lake on OneLake** reads Delta tables from one or more Fabric items. It never falls back to DirectQuery. If it cannot load data, the query fails.
- **Direct Lake on SQL analytics endpoints** reads one Fabric item through its endpoint. When it cannot load a table directly, it **falls back to DirectQuery**: for example when the table is a SQL view, or when the warehouse uses SQL-based granular access control. Fallback queries are slower.

The model property **Direct Lake behavior** controls fallback for the SQL kind:

- `Automatic` (default): silently fall back to `DirectQuery`. Reports keep working, more slowly.
- `DirectLakeOnly`: fail with an error instead. Use it in development to find fallbacks.
- `DirectQueryOnly`: always use `DirectQuery`, to measure the difference.

Guardrails depend on the SKU. For each table, Fabric limits Parquet files, row groups and rows. Over a guardrail, framing can fail (or, on the SQL kind, queries fall back). From Learn's table:

SKU	Parquet files per table	Rows per table	Max memory
F2 to F32	1,000	300 million	3 to 10 GB
F64	5,000	1.5 billion	25 GB
F512 and above	10,000	12 to 24 billion	200 to 400 GB

Small files hit the file guardrail first. Compact gold tables with `OPTIMIZE` (lesson A3-L3), and give Direct Lake tables V-Order (lesson C5-L2).

Why it matters

A sales model on an F32 capacity was fast in testing. After six months of hourly appends, the fact table held 2,400 small Parquet files, above the F32 guardrail of 1,000. The model used Direct Lake on the SQL endpoint with the Automatic behaviour, so queries quietly fell back to `DirectQuery`. Reports took 40 seconds instead of two. Nobody saw an error. A nightly `OPTIMIZE` kept the table at a few dozen files and restored Direct Lake.

Worked example: weak code

- 1 Model: sales (Direct Lake on SQL analytics endpoint), behaviour: Automatic
- 2 Table fact_sales: 2,400 Parquet files, appended hourly, never optimised
- 3 Table v_sales_clean: SQL view over fact_sales
- 4 Capacity: F32

1. **Line 1.** Automatic hides both problems: queries silently fall back.
2. **Line 2.** Above the F32 guardrail of 1,000 files per table.
3. **Line 3.** A SQL view always forces `DirectQuery` fallback on the SQL kind.

Worked example: strong code

- 1 Model: sales (Direct Lake on SQL analytics endpoint), behaviour: `DirectLakeOnly` in dev, Automatic in prod
- 2 Table fact_sales: nightly `OPTIMIZE`, V-Order on, about 40 Parquet files

- 2 Table gold.sales_clean: Delta table written by Spark, replacing the SQL view
- 3 Capacity: F32, fact rows 120 million (limit 300 million)
- 5 Check: framing failures reviewed in refresh history each morning

1. **Line 1.** Development fails loudly on fallback. Production keeps working if one slips through.
2. **Line 3.** Materialise the view's logic as a Delta table so Direct Lake reads it directly.
3. **Line 4.** Row count checked against the SKU's guardrail.

Direct Lake stays fast when tables are Delta (not views), compact, within the SKU's guardrails, and fallback is made visible in development.

Recap card

- **Direct Lake refresh** Framing: records Delta table versions in seconds. No data copy.
- **Direct Lake on OneLake vs on SQL** OneLake: never falls back. SQL: falls back to DirectQuery for views and SQL-based security.
- **Direct Lake behavior values** Automatic (default, silent fallback), DirectLakeOnly (error), DirectQueryOnly (always DirectQuery).
- **F64 per-table guardrails** 5,000 Parquet files, 1.5 billion rows, 25 GB max memory.

Practice

Exercise 1. Multiple choice (level 1)

What does a refresh of a Direct Lake semantic model do?

- a) Reloads all data into the model, like Import.
- b) Frames the model: records which Delta table versions queries will use.
- c) Rewrites the Delta tables.
- d) Switches the model to DirectQuery.

Why?

- r1) Framing copies metadata only, takes seconds, and sets the point in time queries see.
- r2) Direct Lake models never refresh.

Exercise 2. Multiple choice (level 2)

A Direct Lake on SQL analytics endpoint model includes a SQL view. The Direct Lake behavior property is Automatic. What happens when a report queries the view?

- a) The query fails.
- b) The query falls back to DirectQuery and runs more slowly.
- c) The view is materialised automatically.
- d) The view is read directly from OneLake.

Why?

- r1) Views cannot be loaded directly from Delta, and Automatic falls back silently.
- r2) Views are always faster than tables.

Exercise 3. Fill the gap (level 2)

Set the Direct Lake behaviour so that development fails loudly on fallback, and production keeps working.

```
1 Development model: Direct Lake behavior = ____ (1)
2 Production model: Direct Lake behavior = ____ (2)
```

Exercise 4. Write the function (level 3)

Write `guardrail_problems(sku: str, parquet_files: int, rows_millions: float) -> list[str]` using these limits from the lesson: F2 to F32 allow 1,000 files and 300 million rows per table, F64 allows 5,000 and 1,500, F128 5,000 and 3,000, F256 5,000 and 6,000. Return `"files"` and/or `"rows"` for each limit exceeded, in that order. Raise `KeyError` for other SKUs.

Starter code:

```
1 def guardrail_problems(sku, parquet_files, rows_millions):
2     return []
```

Exercise 5. Spot the bug (level 4)

A gold table read by Direct Lake on an F32 grows by one small file per hour and has passed the file guardrail. Tap the line that causes the build-up, and rewrite the job so the table is compacted after the append.

```
1 def publish_hourly(df, table):
2     """Append the latest hour to a Direct Lake gold table."""
3     df.repartition(1).write.mode("append").saveAsTable(table)
4     return None
```

Write down the line number of the bug. Then write the fix.

Sources

- Direct Lake on OneLake never falls back; Direct Lake on SQL falls back to DirectQuery for SQL views and SQL-based granular access control; framing copies metadata in seconds; guardrails per SKU table. (verified: <https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview> (updated 2026-09-02))
- Direct Lake behavior values Automatic (default), DirectLakeOnly and DirectQueryOnly; framing can fail when a table exceeds guardrails. (verified: <https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-how-it-works> (updated 2025-09-22))

Track C best-practice checklist

- ☐ Check the status (GA or preview) of every Fabric feature before you depend on it.
- ☐ One copy of each dataset. Use shortcuts or mirroring before you copy.
- ☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.
- ☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.
- ☐ Semantic models use star schemas. DAX measures state their filter context.
- ☐ Direct Lake models are checked for fallback to DirectQuery.
- ☐ Every Copilot output is read, run on known data and compared against a second method.
- ☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.
- ☐ Items live in Git. Changes move through deployment pipelines.
- ☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C1-L1 Tenant, capacity, workspace and item

C1-L1 exercise 1

Hints. (1) What does a capacity provide? (2) Compute, measured in a unit. (3) Capacity units.

Answer. b) The capacity has 32 capacity units (CUs) of compute. Because:
r1) For F SKUs the number equals the capacity units: F2 is 2 CUs, F64 is 64.
F SKUs are named by their capacity units. F32 provides 32 CUs.

Common wrong answers.

- a: Capacities are not sized by user count. The number is compute in CUs.
- c: OneLake storage is billed separately from capacity compute.
- d: The SKU does not cap workspaces in this way. It measures compute.
- b|r2: Right answer, wrong reason. The number is exactly the CU count.

C1-L1 exercise 2

Hints. (1) Which level holds organisation-wide settings? (2) Which level provides compute to workspaces? (3) Items live in workspaces.

Answer.

1. Tenant
2. Capacity
3. Workspace
4. Item, such as a lakehouse

Tenant settings apply to all capacities. Each workspace draws compute from its capacity. Items live in workspaces.

Common wrong answers.

- A workspace is assigned to a capacity, which provides its compute. Capacity sits above workspace.
- Capacities belong to a tenant, where tenant settings apply to everything.

C1-L1 exercise 3

Hints. (1) Which SKU lets free-licence users view Power BI content? (2) What happens when two workloads share one pool of CUs? (3) Size and isolation.

Answer.

- F64 or larger, because smaller F SKUs require each viewer to hold a Pro, PPU or trial licence.
- Production only, so experiments cannot use up the compute the report needs.

Licensing and isolation both point to a dedicated F64 or larger for this report.

Common wrong answers.

- sku = f16: Below F64, viewers need Pro, Premium Per User or trial licences. 200 free-licence users could not open the report.
- share = sandbox: A heavy notebook in the sandbox consumes the same CUs and can slow or throttle the report.

C1-L1 exercise 4

Hints. (1) Build a mapping from each valid SKU name to its CUs. (2) Check membership in the mapping, then the hours. (3) Multiply and return a float.

Answer.

```
1 F_SKUS: dict[str, int] = {f"F{n}": n for n in (2, 4, 8, 16, 32, 64, 128,
2     256, 512, 1024, 2048)}
3
4 def cu_hours(sku: str, hours: float) -> float:
5     """Capacity-unit hours for running an F SKU capacity.
6
7     Args:
8         sku: F SKU name such as "F64".
9         hours: Hours the capacity runs.
10
11     Returns:
12         CUs multiplied by hours.
13
14     Raises:
15         ValueError: For an unknown SKU or negative hours.
16     """
17     if sku not in F_SKUS:
18         raise ValueError(f"unknown F SKU: {sku!r}")
19     if hours < 0:
20         raise ValueError("hours must not be negative")
21     return float(F_SKUS[sku] * hours)
```

An explicit list of SKUs rejects typos such as F3 or f64 instead of pricing a capacity that does not exist.

C1-L1 exercise 5

Hints. (1) Was the afternoon's compute already used? (2) What happens to usage Fabric smoothed over the coming hours? (3) It is settled at pause.

Answer. b) Smoothed and overage usage still owed is added to the Azure bill at once, and items on the capacity stop working until it resumes. Because: r1) Fabric smooths usage over time. Pausing settles what is still owed and makes the capacity's items unavailable.

Pausing saves money from then on, but the usage already smoothed forward is billed at once, and every item on the capacity stops until it resumes.

Common wrong answers.

- a: The work already happened. Pausing settles the remaining smoothed and overage usage on the bill.
- c: A paused capacity is not operational. Its items stop until you resume it.
- d: Pausing affects compute. It does not delete OneLake data.
- b|r2: Right answer, wrong reason. There is no double rate. Pausing settles usage already incurred.

C1-L2 OneLake, one lake per tenant

C1-L2 exercise 1

Hints. (1) Can you create a second OneLake? (2) What is a workspace in OneLake terms? (3) A folder.

Answer. b) One for the whole tenant Because: r1) Every tenant has a single OneLake. Workspaces organise it; they do not create new lakes.

One tenant, one OneLake. Workspaces and items are folders inside it.

Common wrong answers.

- a: You cannot create more OneLakes. Business units get workspaces and domains inside the one lake.
- c: Capacities provide compute. Storage lives in the tenant's single OneLake.
- d: Each workspace is a folder in OneLake, not a separate lake.
- b|r2: Right answer, wrong reason. It is a platform design, not a licensing unit.

C1-L2 exercise 2

Hints. (1) The workspace goes before the @. (2) The item name carries its type in lower case. (3) Tables for managed Delta tables.

Answer.

```
1 abfss://sales_prod@onelake.dfs.fabric.microsoft.com/lh_sales.lakehouse/
  Tables/orders
```

Workspace before @, then `item.itemtype`, then `Tables/<table>`.

Common wrong answers.

- `g3 = Files`: Files holds unmanaged files. Managed Delta tables live under Tables.
- `g1 = lh_sales`: The part before @ is the workspace, not the item.

C1-L2 exercise 3

Hints. (1) What character does the workspace name contain? (2) The abfs URI does not allow it. (3) Use the GUIDs, which never change.

Answer. The bug is on line 1.

Fixed version:

```
1 WORKSPACE_ID = CONFIG["workspace_id"]
2 LAKEHOUSE_ID = CONFIG["lakehouse_id"]
3 path = f"abfss://{WORKSPACE_ID}@onelake.dfs.fabric.microsoft.com/
  {LAKEHOUSE_ID}/Tables/orders"
```

The space in the workspace name breaks the abfs URI. Renaming the workspace would fix this job and break everything else that uses the old name. GUIDs fix both.

Common wrong answers.

- Line 3: The format of line 3 is right for names. The value it receives on line 1 is the problem.

C1-L2 exercise 4

Hints. (1) Check the item type against a fixed set. (2) Reject spaces in workspace and item names. (3) `relative.lstrip('/')` avoids a double slash.

Answer.

```
1 ONELAKE_HOST = "onelake.dfs.fabric.microsoft.com"
2 ITEM_TYPES = frozenset({"lakehouse", "warehouse"})
3
4
5 def onelake_path(workspace: str, item: str, item_type: str, relative:
  str) -> str:
6     """abfss path to data in a OneLake item, addressed by name.
```

```

7
8     Args:
9         workspace: Workspace name, without spaces.
10        item: Item name, without spaces.
11        item_type: "lakehouse" or "warehouse".
12        relative: Path inside the item, such as Tables/orders.
13
14    Returns:
15        The abfss URI.
16
17    Raises:
18        ValueError: For an unknown item type or names with spaces.
19    """
20    if item_type not in ITEM_TYPES:
21        raise ValueError(f"unsupported item type: {item_type!r}")
22    if " " in workspace or " " in item:
23        raise ValueError("abfs URIs do not allow spaces; use GUIDs
24    instead")
25    return f"abfss://{workspace}@{ONELAKE_HOST}/{item}.{item_type}/{
relative.lstrip('/')}"

```

Validating inputs turns a confusing URI error at read time into a clear message at build time.

C1-L2 exercise 5

Hints. (1) What changes and what does not? (2) Which endpoint guarantees the region? (3) Does Databricks need a copy to read Delta?

Answer.

- GUIDs survive renames and avoid URI problems with special characters.
- The regional endpoint keeps requests in the workspace's region, which the residency rule needs.
- OneLake speaks ADLS APIs and stores open Delta tables, so the job reads them without a second copy.

Stable identifiers, a regional endpoint, and no copies.

Common wrong answers.

- address = names: Names change, and names with spaces break abfs URIs.
- endpoint = global: Microsoft Learn warns that the global endpoint can route data out of your region.
- copy = copy_nightly: A nightly copy is a second dataset to govern and keep in sync. Read the Delta tables where they are.

C1-L3 Workspace roles, granted to groups

C1-L3 exercise 1

Hints. (1) Which rights sit only at the top of the role table? (2) Workspace-level settings. (3) Admin.

Answer. a) Admin only Because: r1) Connecting Git, like deleting the workspace or creating a workspace identity, is reserved for Admins.

Only Admins connect a workspace to Git, create a workspace identity, and update or delete the workspace.

Common wrong answers.

- b: Members can add people and reshare, but connecting Git is an Admin right.
- c: Contributors create and change items, but cannot connect the workspace to Git.
- d: Viewers cannot change anything in the workspace.
- a|r2: Right answer, wrong reason. It is a permission boundary, not a licence.

C1-L3 exercise 2

Hints. (1) Which two read rights does the role table list? (2) Which one does Viewer have? (3) ReadData, the T-SQL one.

Answer. b) It fails: Viewers read lakehouse data with T-SQL, not through Spark or OneLake APIs. Because: r1) Viewer includes ReadData (T-SQL) but not ReadAll (Spark and OneLake APIs).

Point Viewers to the SQL analytics endpoint, or grant ReadAll on the item if Spark access is really needed.

Common wrong answers.

- a: Viewer grants T-SQL reads only. Spark reads need ReadAll, which starts at Contributor.
- c: There is no row limit tied to the role. The read is refused.
- d: Viewers can open and read notebooks. They cannot run them or read data through Spark.
- b|r2: Right answer, wrong reason. Spark is available. The Viewer role lacks ReadAll.

C1-L3 exercise 3

Hints. (1) Who manages, who writes, who reads? (2) Least privilege. (3) Groups, not individuals.

Answer.

- Platform team: Admin, as a group, so management does not depend on one person.
- Deployment identity: Contributor, the only writer in production.
- Analysts: Viewer. They query with T-SQL and use reports.

One managing group, one automated writer, and readers as Viewers.

Common wrong answers.

- analysts = `contributor`: Contributors can change and delete every item in production. Analysts need to read.
- deploy = `admin`: Deployment needs to write items, not manage people or delete the workspace. Contributor is enough.
- platform = `viewer`: Someone must be able to manage the workspace. Give that right to a group.

C1-L3 exercise 4

Hints. (1) A dictionary from action to role. (2) Spark and OneLake reads (ReadAll) start at Contributor. (3) Indexing a dict raises `KeyError` for unknown keys.

Answer.

```
1 MIN_ROLE: dict[str, str] = {
2     "connect_git": "Admin",
3     "delete_workspace": "Admin",
4     "add_members": "Member",
5     "run_notebook": "Contributor",
6     "spark_read": "Contributor",
7     "tsql_read": "Viewer",
8 }
9
10
11 def min_role(action: str) -> str:
12     """Lowest Fabric workspace role that allows an action.
13
14     Raises:
15         KeyError: For an action not in the table.
16     """
17     return MIN_ROLE[action]
```

Encoding the role table as data lets access requests be checked automatically, and an unknown action fails loudly instead of defaulting to a guess.

C1-L3 exercise 5

Hints. (1) Which line grants write access? (2) Do analysts need to change production items? (3) Also replace the individual with a group.

Answer. The bug is on line 4.

Fixed version:

```
1 workspace: sales_prod
2 roles:
3   Admin: [grp-fabric-platform-admins]
4   Contributor: [sp-sales-deployment]
5   Viewer: [grp-sales-analysts, grp-sales-readers]
```

Analysts as Contributors can change or delete production items. Make them Viewers, keep automation as the only writer, and replace the individual with a group.

Common wrong answers.

- Line 5: An individual assignment breaks the groups rule, but it only grants read access. Line 4 gives write access to production.
- Line 3: An admin group is the right pattern.

C2-L1 Lakehouse Tables and Files, and schemas

C2-L1 exercise 1

Hints. (1) Which format does the endpoint read? (2) Which folder holds managed tables? (3) Delta, in Tables.

Answer. b) Nothing. Only Delta tables in Tables appear in the endpoint. Because: r1) The endpoint exposes Delta tables only. Parquet and CSV cannot be queried through it.

Only Delta tables reach the SQL engines. Files is for raw and non-Delta data.

Common wrong answers.

- a: Files is not scanned for tables. Write a Delta table to Tables with saveAsTable.
- c: Refreshing metadata picks up Delta tables. It never turns a Parquet file into a table.
- d: Fabric does not convert it. You write the Delta table yourself.

- `b|r2`: Right answer, wrong reason. Files is visible. It is just not tables.

C2-L1 exercise 2

Hints. (1) What fills in the missing parts of the name? (2) The default lakehouse and the default schema. (3) `dbo`.

Answer. `a) lh_survey.dbo.respondents` Because: `r1)` A bare table name resolves to the `dbo` schema of the default lakehouse.

`respondents` resolves to `lh_survey.dbo.respondents`. If the table lives in `sales`, name the schema.

Common wrong answers.

- `b`: Spark does not search schemas. Missing parts come from defaults: default lakehouse, `dbo` schema.
- `c`: Omitted parts are allowed and filled from defaults.
- `d`: Nothing is unioned. One table is resolved.
- `a|r2`: Right answer, wrong reason. Spark reads any schema you name. `dbo` is only the default.

C2-L1 exercise 3

Hints. (1) The four-part name is `workspace.lakehouse.schema.table`. (2) The third part is a schema. (3) The `employees` table lives in schema `hrm` in Microsoft's example, or `dbo` by default.

Answer.

```
1 SELECT e.name, d.dept_name
2 FROM ops_ws.hr_lakehouse.hrm.employees AS e
3 JOIN global_ws.corporate_lakehouse.company.departments AS d
4   ON e.deptno = d.deptno
```

`workspace.lakehouse.schema.table` reaches any table you have permission to read, across lakehouses and workspaces.

Common wrong answers.

- `g1 = Tables`: `Tables` is a OneLake folder, not a schema. The third part is the schema name, such as `dbo` or `hrm`.

C2-L1 exercise 4

Hints. (1) Read with `.schema(RESPONDENT_SCHEMA)` and the header option. (2) Write with `.write.mode("append").saveAsTable(table)`. (3) Return the `DataFrame`.

Answer.

```

1 from pyspark.sql import DataFrame, SparkSession
2 from pyspark.sql import types as T
3
4 RESPONDENT_SCHEMA = T.StructType([
5     T.StructField("respondent_id", T.StringType(), False),
6     T.StructField("wave", T.IntegerType(), False),
7     T.StructField("weight", T.DoubleType(), True),
8 ])
9
10
11 def load_delivery(spark: SparkSession, path: str, table: str) ->
    DataFrame:
12     """Read a CSV delivery and append it to a managed Delta table.
13
14     Args:
15         spark: Active session.
16         path: CSV path, usually under Files/landing.
17         table: Target table, ideally lakehouse.schema.table.
18
19     Returns:
20         The DataFrame that was read.
21     """
22     df = spark.read.option("header",
23                             True).schema(RESPONDENT_SCHEMA).csv(path)
24     df.write.mode("append").saveAsTable(table)
25     return df

```

A managed Delta table in Tables serves Spark, the SQL analytics endpoint and Direct Lake from one copy.

C2-L1 exercise 5

Hints. (1) Which area holds non-Delta files? (2) Which area do SQL and Power BI read? (3) Schemas organise tables by domain.

Answer.

- Raw deliveries in Files, as received.
- Cleaned and reporting data as Delta tables in Tables, under schemas such as survey_silver and survey_gold.

Files for raw, Tables for Delta, and schemas to keep the tables organised.

Common wrong answers.

- raw = tables_dbo: CSV files under Tables are not Delta tables. They end up in the Unidentified area.
- gold = files: Power BI's Direct Lake and the SQL endpoint read Delta tables in Tables, not files.
- clean = tables_dbo: It works, but everything lands in one flat schema. Named schemas keep domains apart as the lakehouse grows.

C2-L2 Warehouse T-SQL, and where it differs from SQL Server

C2-L2 exercise 1

Hints. (1) What does NOT ENFORCED mean? (2) Does the engine check the key on insert? (3) No.

Answer. b) Both rows are stored. The key is not enforced. Because: r1) NOT ENFORCED keys are optimiser hints. The engine does not check them.

Both rows land. Check uniqueness after each load with a GROUP BY ... HAVING COUNT(*) > 1 query.

Common wrong answers.

- a: That is SQL Server behaviour for enforced keys. Fabric Warehouse keys are NOT ENFORCED.
- c: INSERT never replaces rows. You would need MERGE for upsert behaviour.
- d: Nothing locks. Both rows are simply stored.
- b|r2: Right answer, wrong reason. No background job removes duplicates. You must check.

C2-L2 exercise 2

Hints. (1) Not clustered. (2) Not checked by the engine. (3) NONCLUSTERED and NOT ENFORCED.

Answer.

```
1 ALTER TABLE dbo.dim_customer
2   ADD CONSTRAINT pk_dim_customer PRIMARY KEY NONCLUSTERED (customer_id)
   NOT ENFORCED;
```

PRIMARY KEY NONCLUSTERED (...) NOT ENFORCED is the only accepted form.

Common wrong answers.

- g1 = CLUSTERED: Fabric Warehouse supports primary keys only as NONCLUSTERED.

C2-L2 exercise 3

Hints. (1) Two SQL Server types have no Fabric Warehouse equivalent. (2) One for money, one for Unicode text. (3) decimal and varchar replace them.

Answer. The bug is on line 4, 5.

Fixed version:

```

1 CREATE TABLE dbo.fact_payment (
2     payment_id BIGINT NOT NULL,
3     customer_id INT NOT NULL,
4     amount DECIMAL(19, 4) NOT NULL,
5     note VARCHAR(500),
6     paid_at DATETIME2(6) NOT NULL
7 );

```

money and nvarchar are not supported. Use DECIMAL(19, 4) and VARCHAR(500). Varchar uses a UTF-8 collation, so it stores Unicode text.

Common wrong answers.

- Line 6: datetime2 with up to 6 fractional digits is supported.
- Line 2: BIGINT is supported.

C2-L2 exercise 4

Hints. (1) Count rows per customer_id. (2) Filter groups, not rows. (3) HAVING COUNT(*) > 1.

Answer.

```

1 SELECT customer_id, COUNT(*) AS n
2 FROM dbo.dim_customer
3 GROUP BY customer_id
4 HAVING COUNT(*) > 1;

```

With keys unenforced, this query is the guard. Any row it returns means a duplicate got in.

C2-L2 exercise 5

Hints. (1) What no longer enforces uniqueness? (2) Which feature is not supported at all? (3) Use the documented alternatives.

Answer.

- Keys become hints, so add a duplicate check that fails the load.
- Triggers are not supported. Do the audit write explicitly in the loading procedure.
- Map each type to its documented alternative, keeping precision and scale.

A port succeeds when every lost guarantee is replaced by an explicit step.

Common wrong answers.

- pk = trust: Sources resend files and pipelines rerun. Without a check, duplicates load silently.
- trigger = keep: Fabric Warehouse does not support triggers. The CREATE TRIGGER statement fails.

- `types = string`: Strings lose numeric semantics and cost storage. Use decimal for amounts.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

C2-L3 exercise 1

Hints. (1) The endpoint is described as read-only. Read-only for what? (2) It blocks changes to table data, not every kind of object. (3) Views, functions and stored procedures that read are allowed.

Answer. b) You can create a view over lakehouse tables in it. Because: r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.

The endpoint is read-only for table data, but you can create views, functions and stored procedures in it. Fabric creates it with the lakehouse, and it reads the same Delta tables in OneLake.

Common wrong answers.

- a: The endpoint is read-only for table data. Write with Spark in the lakehouse, or move the table to a warehouse.
- c: Fabric creates the endpoint automatically with every lakehouse.
- d: It reads the same Delta tables in OneLake. There is no second copy. It syncs metadata, not data.
- b|r2: Right choice, wrong reason. A view is a T-SQL object in the endpoint. The Delta tables stay unchanged.

C2-L3 exercise 2

Hints. (1) Ask who writes the data, and in which language. (2) Spark writers and files: one item. T-SQL writers: another. (3) T-SQL readers of lakehouse data need no new item at all.

Answer.

- Raw files and PySpark: **Lakehouse**. Spark writes, and files sit next to tables.
- T-SQL writes across several tables: **Warehouse**, for DML and multi-table transactions.
- T-SQL reads of lakehouse data, plus views: **SQL analytics endpoint**. No copy needed.

Spark writers use a lakehouse. T-SQL writers use a warehouse. T-SQL readers of lakehouse data use the endpoint that comes with the lakehouse.

Common wrong answers.

- finance = `lakehouse`: The finance team writes with T-SQL. A lakehouse accepts writes from Spark, and its endpoint is read-only. Use a warehouse.
- finance = `endpoint`: The endpoint cannot change table data, so the stored procedures would fail on their first UPDATE or INSERT.
- raw = `warehouse`: A warehouse does not hold raw files, and the team writes with PySpark. Use a lakehouse.
- adhoc = `warehouse`: Copying the lakehouse tables into a warehouse makes a second copy to keep in sync. The endpoint reads them where they are.

C2-L3 exercise 3

Hints. (1) Which statements change table data? (2) The endpoint accepts reads and objects that read. (3) UPDATE on a lakehouse table fails through the endpoint.

Answer. The bug is on line 7.

Fixed version:

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 -- Fix wave W00 in the lakehouse with Spark, not here.
```

Line 7 is an `UPDATE` on a lakehouse table, which the read-only endpoint rejects. Make that correction in a Spark notebook against the lakehouse, and keep only the view here.

Common wrong answers.

- Line 2: Creating a view is allowed on the endpoint. Look for a statement that changes table data.
- Line 3: The division could hit zero, but that is not what stops this script. Look for a write.

C2-L3 exercise 4

Hints. (1) Aggregate both sums per wave, then divide. (2) `NULLIF(x, 0)` returns NULL when x is 0, and dividing by NULL gives NULL. (3) `SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct`, then `GROUP BY wave_id`.

Answer.

```

1 CREATE VIEW dbo.v_wave_reach AS
2 SELECT
3     wave_id,
4     SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
5 FROM dbo.nat_metrics
6 GROUP BY wave_id;

```

Sum both columns per wave and divide by `NULLIF(SUM(weighted_base), 0)`. A zero base becomes NULL, so the division returns NULL instead of an error.

C2-L3 exercise 5

Hints. (1) Three-part names are `item.schema.table`. (2) The first part names the item that holds the table. (3) The table sits in the lakehouse `lh_research`.

Answer.

```

1 INSERT INTO dbo.wave_summary (wave_id, weighted_base)
2 SELECT wave_id, SUM(weighted_base)
3 FROM lh_research.dbo.nat_metrics
4 GROUP BY wave_id;

```

A warehouse query reads a lakehouse table in the same workspace with `lh_research.dbo.nat_metrics`. The data stays in one place.

Common wrong answers.

- `g1 = wh_finance`: `wh_finance` is the warehouse you are in. The table lives in the lakehouse, so name the lakehouse.
- `g1 = OneLake`: There is no database called OneLake. The first part of a three-part name is the item name: the lakehouse.

C2-L3 exercise 6

Hints. (1) The query failed first and worked later, with no change in between. (2) How does the endpoint learn that a new table exists? (3) Look at the metadata sync between the lakehouse and its endpoint.

Answer. b) The endpoint had not yet synced the new table's metadata. Because: r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.

The endpoint syncs lakehouse metadata automatically, usually in under a minute, but the lag varies. When a later step depends on a new table, add a metadata refresh step (portal, REST API) or a retry before the query. Check the current refresh options on Microsoft Learn.

Common wrong answers.

- `a`: The query worked an hour later without any change to the table, so the format was fine.

- **c**: The sync runs automatically. A manual refresh speeds it up, but it is not required every time.
- **d**: Permissions do not change on a timer. The delay points to the metadata sync.
- **b|r2**: Right choice, wrong reason. The endpoint does not copy data. It syncs metadata about the Delta tables.

C3-L1 OneLake shortcuts, and the delete that goes through

C3-L1 exercise 1

Hints. (1) Is the shortcut the data or a pointer? (2) What does removing a pointer do? (3) Contrast it with deleting inside the pointer.

Answer. **b**) It stays. Deleting a shortcut deletes only the shortcut object. Because: **r1**) Shortcuts do not cascade deletes. The shortcut is an independent object.

The shortcut goes, the data stays. Deleting a file or folder inside the shortcut is different.

Common wrong answers.

- **a**: Removing the shortcut removes only the link. Deleting files inside it is what reaches the target.
- **c**: There is no copy to recycle. The data was never in OneLake.
- **d**: Nothing is copied when you delete a shortcut.
- **b|r2**: Right answer, wrong reason. Deleting files inside a shortcut can delete ADLS data.

C3-L1 exercise 2

Hints. (1) Where do the files inside a shortcut live? (2) In the target. (3) Your permissions there decide.

Answer. **b**) The 2023 folder is deleted in the target storage account. Because: **r1**) Deleting a file or folder within a shortcut deletes it in the target when you have permission there.

Inside a shortcut you are working on the target. Deletes go through.

Common wrong answers.

- **a**: There is no hidden view. The folder is the target folder.
- **c**: Shortcuts are not read-only when your identity can write to the target.

- `d`: The shortcut still works. It just points at data that no longer includes 2023.
- `b|r2`: Right answer, wrong reason. The delete happens immediately, not on a schedule.

C3-L1 exercise 3

Hints. (1) Which area holds managed tables? (2) Which format does the endpoint read? (3) Tables and Delta.

Answer.

```
1 Create the shortcut under Tables, pointing at data in Delta format.
```

A shortcut under Tables that points at Delta data is recognised as a table.

Common wrong answers.

- `g1 = Files`: Shortcuts under Files can hold any format, but they do not become tables.

C3-L1 exercise 4

Hints. (1) Check shortcuts first and refuse early. (2) Compare against `root + "/"` so partner does not match `partner_old`. (3) Allowlist owned prefixes with the trailing slash.

Answer.

```
1 OWNED_PREFIXES = ("Files/landing/", "Files/tmp/")
2
3
4 def safe_to_delete(path: str, shortcut_roots: set[str]) -> bool:
5     """True only for owned paths that are not under a shortcut.
6
7     Args:
8         path: Lakehouse-relative path, such as Files/landing/w14.csv.
9         shortcut_roots: Paths of every shortcut in the lakehouse.
10
11     Returns:
12         Whether a clean-up job may delete the path.
13     """
14     for root in shortcut_roots:
15         base = root.rstrip("/")
16         if path == base or path.startswith(base + "/"):
17             return False
18     return path.startswith(OWNED_PREFIXES)
```

Refusing anything under a shortcut, and allowing only named folders, makes a clean-up job unable to reach someone else's data.

C3-L1 exercise 5

Hints. (1) Copy or point? (2) Where do shortcuts become tables? (3) What stops a delete from going through?

Answer.

- A shortcut avoids a second copy and a pipeline to maintain.
- Under Tables, Delta data becomes a table for SQL and Direct Lake.
- Caching cuts egress costs for repeated reads from S3.
- A read-only identity means no delete inside the shortcut can reach the bucket.

Point, place it where it becomes a table, cache cross-cloud reads, and connect read-only.

Common wrong answers.

- access = `copy`: A copy doubles storage and needs a pipeline and monitoring. A shortcut reads in place.
- writes = `owner`: With write access, deleting inside the shortcut deletes marketing's data.
- location = `files`: Under Files it will not be recognised as a table.

C3-L2 Mirroring operational databases into OneLake

C3-L2 exercise 1

Hints. (1) What do shortcuts point at? (2) Is an Azure SQL database a folder of Delta files? (3) It must be replicated.

Answer. b) Mirroring: it replicates the database into OneLake continuously. Because: r1) Azure SQL stores data in a proprietary format, so Fabric must replicate it, which is what mirroring does.

Databases in proprietary formats come in through mirroring. Open-format data in storage comes in through shortcuts.

Common wrong answers.

- `a`: Shortcuts point at storage (OneLake, ADLS, S3 and similar), not at database engines.
- `c`: A nightly export is a hand-built copy with lag and maintenance. Mirroring does it continuously.
- `d`: There is no storage folder of Delta files to point at. The database must be replicated.

- `b|r2`: Right answer, wrong reason. Shortcuts copy nothing. Mirroring is needed because the source is a database.

C3-L2 exercise 2

Hints. (1) How many CUs does the capacity have? (2) 1 TB free per CU. (3) 45 - 32.

Answer.

```
1 32 13
```

32 TB are free on an F32. The remaining 13 TB of mirrored storage is billed as OneLake storage.

Common wrong answers.

- `64 0`: The example on Learn uses F64. This capacity has 32 CUs, so 32 TB are free.

C3-L2 exercise 3

Hints. (1) The standard catalog views schema. (2) Tables, not views. (3) `INFORMATION_SCHEMA` and `BASE TABLE`.

Answer.

```
1 SELECT TABLE_SCHEMA, TABLE_NAME
2 FROM INFORMATION_SCHEMA.TABLES
3 WHERE TABLE_TYPE = 'BASE TABLE';
```

`INFORMATION_SCHEMA.TABLES` lists the tables the endpoint exposes. Compare it with the source list.

Common wrong answers.

- `g2 = VIEW`: Views are not mirrored tables. Filter on `BASE TABLE`.

C3-L2 exercise 4

Hints. (1) Loop over the source tables. (2) `mirror.get(table)` is `None` when absent, so one comparison covers both cases. (3) Sort the result.

Answer.

```
1 def missing_from_mirror(source: dict[str, int], mirror: dict[str, int]) -
  > list[str]:
2     """Source tables that are absent from the mirror or differ in row
    count.
3
4     Args:
5         source: schema.table to row count in the source database.
6         mirror: schema.table to row count on the mirror's endpoint.
7
8     Returns:
```

```

9         Sorted table names that need attention.
10        """
11        return sorted(t for t, n in source.items() if mirror.get(t) != n)

```

One comparison, `mirror.get(t) != n`, catches both missing tables and partial replication.

C3-L2 exercise 5

Hints. (1) Is there a table limit? (2) Which key types block mirroring? (3) How would you notice a missing table?

Answer.

- Mirror what reporting needs, within the 1,000-table limit.
- Tables with `datetime2(7)` keys cannot be mirrored. Plan for them explicitly.
- Reconcile daily, because ineligible tables fail quietly.

Plan within the documented limits, then verify with data, not with status pages.

Common wrong answers.

- `scope = all`: Azure SQL Database mirroring supports at most 1,000 tables.
- `keys = ignore`: Those tables are not eligible. They will simply be missing.
- `check = trust`: A table that was never eligible does not show as failing. Compare lists and counts.

C3-L3 Mirroring Azure Databricks Unity Catalog into Fabric

C3-L3 exercise 1

Hints. (1) What format are Unity Catalog managed tables stored in? (2) Can Fabric read that format directly? (3) Metadata plus shortcuts.

Answer. b) Only the catalog structure. Data is read through shortcuts. Because: r1) Unity Catalog tables are already open Delta files, so Fabric mirrors metadata and reads the files in place.

Only the catalog structure is mirrored. The data stays in the Databricks storage account.

Common wrong answers.

- a: That is database mirroring for proprietary sources. Unity Catalog mirroring has no data movement.

- **c**: There are no snapshots. Fabric reads the current Delta files through shortcuts.
- **d**: Fabric engines read the Delta files directly. No Databricks compute runs the queries.
- **b** | **r2**: Right answer, wrong reason. It is about format: Delta can be read in place.

C3-L3 exercise 2

Hints. (1) Check the list of unsupported table types. (2) RLS/CLM policies. (3) The table is left out.

Answer. **c**) It is not mirrored: tables with RLS or CLM policies are unsupported. Because: **r1**) The limitations page lists tables with RLS/CLM policies as unsupported.

Filtered and masked tables are excluded. Decide what to publish for Fabric users instead.

Common wrong answers.

- **a**: Unity Catalog policies are not carried into Fabric, and such tables are not supported.
- **b**: Fabric does not quietly drop the filter. It does not mirror the table.
- **d**: Views are themselves unsupported for this mirroring.
- **c** | **r2**: Right answer, wrong reason. The table type is unsupported, not read-only.

C3-L3 exercise 3

Hints. (1) Scope before you connect. (2) The connection's rights cap what is readable. (3) Access, then verify.

Answer.

1. Choose tables safe for the whole Fabric audience, in a dedicated schema
2. Create a connection identity with SELECT on that schema only
3. Create the mirrored Azure Databricks item for that schema
4. Grant Fabric access with workspace roles, SQL permissions or model RLS
5. Compare table lists and row counts with Databricks

Scope, least-privilege connection, mirror, Fabric access, then a data check.

Common wrong answers.

- Mirroring first exposes whatever the connection can read. Decide the scope first.

- Set up access before users arrive. Verification then confirms what they will see.

C3-L3 exercise 4

Hints. (1) Allow a fixed set of table types rather than excluding a list. (2) Also exclude tables with policies. (3) Sort the names.

Answer.

```

1 MIRRORABLE_TYPES = frozenset({"MANAGED", "EXTERNAL"})
2
3
4 def mirror_eligible(tables: list[dict[str, object]]) -> list[str]:
5     """Unity Catalog tables that Fabric can mirror.
6
7     Args:
8         tables: Dicts with name, table_type and has_row_filter_or_mask.
9
10    Returns:
11        Sorted names of eligible tables.
12    """
13    return sorted(
14        str(t["name"])
15        for t in tables
16        if t["table_type"] in MIRRORABLE_TYPES and not
17        t["has_row_filter_or_mask"]
18    )

```

An allowlist of table types plus the policy check predicts which tables will appear, so a missing table is expected rather than a surprise.

C3-L3 exercise 5

Hints. (1) Where do Unity Catalog rules stop applying? (2) Whose credential reads the data? (3) Where does the RLS live?

Answer.

- The filtered table cannot be mirrored, so rebuild the regional rule where users consume the data.
- Scope the connection, because it reads every mirrored table for everyone.
- Keep managers on the report. Direct endpoint access would bypass the model's RLS.

Rebuild the rule where users are, scope the credential, and keep users on the governed path.

Common wrong answers.

- source = `unfiltered_all`: Every manager would see every region. The rule must be rebuilt in Fabric.
- connection = `broad`: The connection's credential serves all queries. Grant it only what the mirror needs.
- users = `viewer`: Viewers can query the endpoint with T-SQL and read rows the model's RLS would hide.

C4-L1 Choosing how data gets into Fabric

C4-L1 exercise 1

Hints. (1) Which layer is this? (2) Which tool tracks incremental state itself? (3) Copy job.

Answer. b) Copy job Because: r1) Copy job supports incremental copy and scheduling natively for bronze ingestion.

Bronze ingestion starts with Copy job: incremental copy without building a pipeline.

Common wrong answers.

- a: Mirroring replicates databases and catalogs. This is file ingestion into bronze.
- c: Dataflow Gen2 is for low-code transformation. Nothing needs transforming yet.
- d: It works, but you build and maintain the incremental state that Copy job tracks for you.
- b|r2: Right answer, wrong reason. Pipelines and shortcuts read ADLS too.

C4-L1 exercise 2

Hints. (1) Replicate, stream, transform or orchestrate? (2) Which tool is event-driven? (3) Which tool coordinates other tools?

Answer.

- Curated database tables: mirroring.
- Low-latency events: Eventstream.
- Heavy transformation: Spark.
- Ordering, branching and alerts across steps: a pipeline.

Each tool has a lane. Stay in it and there is less to build and maintain.

Common wrong answers.

- `erp = copyjob`: A Copy job works, but mirroring replicates the whole database with free replication compute and no schedule to manage.
- `iot = copyjob`: Copy jobs run on schedules. Seconds-level events need Eventstream.
- `orchestrate = spark`: A notebook can call others, but retries, branches and alerts across steps are what pipelines are for.

C4-L1 exercise 3

Hints. (1) Movement first. (2) Transformation next. (3) Orchestration wraps the rest.

Answer.

1. Replicate a whole database or catalog?
2. Copy tables or files into bronze?
3. Events that must arrive within seconds?
4. Transformation needed, and by whom?
5. Several steps that need ordering and alerts?

Choose movement tools, then transformation, then wrap them in orchestration only where needed.

Common wrong answers.

- Starting with a pipeline leads to building copy logic inside it. Choose the movement tools first, then orchestrate them.
- Decide how data arrives before how it is transformed.

C4-L1 exercise 4

Hints. (1) A dictionary from need to tool. (2) Check membership before returning. (3) Put the need in the error message.

Answer.

```
1 TOOL_FOR_NEED: dict[str, str] = {
2     "replicate_database": "Mirroring",
3     "bronze_copy": "Copy job",
4     "low_latency_events": "Eventstream",
5     "code_transform": "Spark notebook",
6     "lowcode_transform": "Dataflow Gen2",
7     "orchestrate": "Pipeline",
8 }
9
10
11 def recommend(need: str) -> str:
```

```

12     """Fabric tool for a data movement or processing need.
13
14     Raises:
15         ValueError: If the need is not in the table.
16     """
17     if need not in TOOL_FOR_NEED:
18         raise ValueError(f"unknown need: {need}")
19     return TOOL_FOR_NEED[need]

```

An explicit table makes the team's convention reviewable, and a clear error beats a silent default.

C4-L1 exercise 5

Hints. (1) Where does the bug live? (2) In hand-written state tracking. (3) Which tool does that for you?

Answer. b) Move the tables to Copy jobs with incremental copy, which track state themselves. Because: r1) Copy job manages the state of the last successful run, so there is no hand-written watermark logic to get wrong.

Removing hand-built incremental state removes the bugs it causes. Copy job tracks the last successful run itself.

Common wrong answers.

- a: Retries repeat the same logic, including its watermark bug.
- c: More runs repeat the off-by-one error more often.
- d: That moves the hand-built state into another tool without removing it.
- b|r2: Right answer, wrong reason. Pipelines can copy incrementally, but you build the state tracking.

C4-L2 Incremental copy with a watermark, or with CDC

C4-L2 exercise 1

Hints. (1) What does a watermark look for? (2) Does a deleted row have a new value? (3) CDC captures deletes.

Answer. b) Nothing. Watermark-based incremental copy cannot detect deletes. Because: r1) A watermark only finds rows with a larger value. A deleted row has no value to find.

Watermarks only see rows that exist with a larger value. Use CDC when deletes matter.

Common wrong answers.

- a: That needs CDC, which captures delete events.

- `c`: Soft deletes come from CDC with SCD Type 2. A watermark sees nothing.
- `d`: The job succeeds. The deleted row just stays in the destination.
- `b|r2`: Right answer, wrong reason. CDC-based incremental copy does replicate deletes.

C4-L2 exercise 2

Hints. (1) Which rows have `rv` above 105? (2) Where does row 3 fall? (3) Below the watermark.

Answer.

```
1 [4]
```

Only row 4 is copied. Row 3 is lost for good unless you reset to a full copy, which is why commit-order columns such as `ROWVERSION`, or `CDC`, are safer.

Common wrong answers.

- `[3, 4]`: Row 3 has `rv` 104, below the watermark 105, so it is never copied. That is the late-row blind spot.

C4-L2 exercise 3

Hints. (1) Which method captures deletes? (2) Which update method keeps one row per key? (3) `CDC` and `Merge`.

Answer.

```
1 Incremental method: CDC
2 Update method: Merge
```

`CDC` replicates inserts, updates and deletes. `Merge` keeps one row per key.

Common wrong answers.

- `g2 = Append`: `Append` adds each changed row again, so a key appears many times.
- `g1 = Watermark`: A watermark cannot see deletes.

C4-L2 exercise 4

Hints. (1) Handle `last_wm` `None` first. (2) Strictly greater than, to avoid re-copying the boundary. (3) Return `([], last_wm)` when nothing is new.

Answer.

```
1 Row = dict[str, int]
2
3
4 def next_batch(rows: list[Row], last_wm: int | None) -> tuple[list[Row],
    int | None]:
```

```

5     """Rows above the watermark, and the new watermark.
6
7     Args:
8         rows: Source rows, each with an integer rv.
9         last_wm: Largest rv copied so far, or None before the first run.
10
11     Returns:
12         The rows to copy and the watermark to store.
13     """
14     batch = [r for r in rows if last_wm is None or r["rv"] > last_wm]
15     if not batch:
16         return [], last_wm
17     return batch, max(r["rv"] for r in batch)

```

Strictly greater avoids duplicates, and the empty-run guard keeps the watermark stable. The late-row gap remains, which is why CDC or ROWVERSION matters.

C4-L2 exercise 5

Hints. (1) Deletes matter here. (2) One row per key. (3) How do you recover missed changes?

Answer.

- CDC captures inserts, updates and deletes, with no watermark blind spots.
- Merge keeps one current row per order.
- Reset to a full copy for the table when you know changes were missed.

CDC plus Merge keeps silver in step with the source, and a per-table reset repairs known gaps.

Common wrong answers.

- method = `wm_modified`: A watermark misses deletes and late rows.
- update = `append`: Append adds each changed order again, breaking one row per key.
- recovery = `wait`: Changes below the stored state are not picked up by later runs.

C4-L3 Pipelines that fail fast, retry sensibly and alert

C4-L3 exercise 1

Hints. (1) It is much longer than most jobs. (2) Half a day. (3) 12 hours.

Answer. c) 12 hours Because: r1) The documented default timeout is 12 hours. Seven days is the maximum you can set.

12 hours by default. Set each activity's timeout close to its real run time, with some headroom.

Common wrong answers.

- a: 30 seconds is the default retry interval. The default timeout is 12 hours.
- d: Seven days is the maximum, not the default.
- b: The default is 12 hours.
- c|r2: Right answer, wrong reason. Timeouts are unrelated to capacity pausing.

C4-L3 exercise 2

Hints. (1) Four branches: success, failure, completion, skip. (2) Completion is not a failure. (3) Either outcome.

Answer. c) Whether the notebook succeeds or fails Because: r1) Upon Completion runs after success or failure. Upon Failure is the failure-only branch.

A failure alert on Upon Completion fires every night. Use Upon Failure.

Common wrong answers.

- a: That is Upon Failure.
- b: That is Upon Success.
- d: That is Upon Skip.
- c|r2: Right answer, wrong reason. It refers to this activity finishing, either way.

C4-L3 exercise 3

Hints. (1) Inside a string, expressions are wrapped in `@{ }`. (2) A whole value that must stay a number starts with `@` alone. (3) `@{ }` and `@`.

Answer.

```
1 Folder path: landing/@{pipeline().parameters.wave}/
2 Max rows:      @pipeline().parameters.max_rows
```

`@{...}` inside text interpolates to a string. A bare `@...` expression keeps the value's type.

Common wrong answers.

- `g2 = @:` String interpolation always returns a string. For a number, use the bare `@` expression.

C4-L3 exercise 4

Hints. (1) A regular expression with fullmatch validates the shape. (2) Check each field's range. (3) Cap the total at $7 \times 24 \times 3600$ seconds.

Answer.

```
1 import re
2
3 TIMEOUT_PATTERN = re.compile(r"(\d+)\.(\d{2}):(\d{2}):(\d{2})")
4 MAX_SECONDS = 7 * 24 * 3600
5
6
7 def parse_timeout(value: str) -> int:
8     """Seconds in a pipeline activity timeout written as D.HH:MM:SS.
9
10    Raises:
11        ValueError: On a bad format, out-of-range fields, or more than
12        seven days.
13    """
14    match = TIMEOUT_PATTERN.fullmatch(value)
15    if match is None:
16        raise ValueError(f"timeout must be D.HH:MM:SS, got {value!r}")
17    days, hours, minutes, seconds = (int(g) for g in match.groups())
18    if hours > 23 or minutes > 59 or seconds > 59:
19        raise ValueError(f"field out of range in {value!r}")
20    total = days * 86400 + hours * 3600 + minutes * 60 + seconds
21    if total > MAX_SECONDS:
22        raise ValueError("timeout exceeds the seven-day maximum")
23    return total
```

A linter that parses timeouts can flag every activity still on the 12-hour default, or set far above its normal run time.

C4-L3 exercise 5

Hints. (1) Which branch lets gold run after a failure? (2) Completion means either outcome. (3) Use Upon Success.

Answer. The bug is on line 3.

Fixed version:

```
1 Activity: nb_silver_orders
2   Timeout: 0.00:30:00
3 Activity: nb_gold_sales depends on nb_silver_orders: Upon Success
4 Activity: email_on_failure depends on nb_silver_orders: Upon Failure
```

Upon Completion runs gold after a failed silver, publishing stale or partial data. Chain data steps on Upon Success.

Common wrong answers.

- Line 4: The alert is correctly on Upon Failure.

- Line 2: A 30-minute timeout is sensible for a 10-minute notebook.

C5-L1 From Databricks notebooks to Fabric notebooks

C5-L1 exercise 1

Hints. (1) Where do Fabric secrets live? (2) Which utility module handles credentials? (3) `notebookutils.credentials`.

Answer. b) `notebookutils.credentials.getSecret("https://kv-sales.vault.azure.net/", "sql-password")` Because: r1) Fabric reads secrets from Azure Key Vault through `notebookutils.credentials`.

`getSecret(vault_url, name)` reads from Azure Key Vault, with your identity's Get permission.

Common wrong answers.

- a: `dbutils` is Databricks-only. Fabric has no secret scopes.
- c: Workspace settings do not hold secrets as environment variables. Use Key Vault.
- d: Spark configuration is not a secret store.
- b|r2: Right answer, wrong reason. It reads from Key Vault at runtime.

C5-L1 exercise 2

Hints. (1) Look at `run()`'s signature. (2) `timeout_seconds` has a default. (3) 90.

Answer. b) It fails after 90 seconds, the default timeout. Because: r1) `run()` has `timeout_seconds` with a default of 90.

Pass `timeout_seconds` sized to the child's real run time with headroom.

Common wrong answers.

- a: Only if you pass a longer `timeout_seconds`.
- c: 12 hours is the pipeline activity default. `notebook.run` defaults to 90 seconds.
- d: `run()` waits for the child and returns its exit value.
- b|r2: Right answer, wrong reason. The limit is time, not job count.

C5-L1 exercise 3

Hints. (1) The Fabric utility module. (2) The name of `run()`'s timeout parameter. (3) `notebookutils` and `timeout_seconds`.

Answer.

```
1 files = notebookutils.fs.ls("Files/landing")
2 value = notebookutils.notebook.run("nb_child", timeout_seconds=600)
```

`notebookutils` replaces `dbutils`, and `timeout_seconds` sets the child's time limit.

Common wrong answers.

- `g1 = dbutils`: `dbutils` does not exist in Fabric.
- `g1 = mssparkutils`: `mssparkutils` still works but is being retired. Use `notebookutils`.

C5-L1 exercise 4

Hints. (1) Check for `dbutils.secrets.first` and `raise`. (2) Replace the specific prefixes, including the trailing dot. (3) Leave anything else alone.

Answer.

```
1 REWRITES: tuple[tuple[str, str], ...] = (
2     ("dbutils.fs.", "notebookutils.fs."),
3     ("dbutils.notebook.", "notebookutils.notebook."),
4     ("mssparkutils.", "notebookutils."),
5 )
6
7
8 def port_line(line: str) -> str:
9     """Rewrite Databricks and legacy Fabric utility calls to
10     notebookutils.
11
12     Raises:
13         ValueError: For secret lookups, which must move to Azure Key Vault
14         by hand.
15     """
16     if "dbutils.secrets." in line:
17         raise ValueError("secrets need a manual change: use
18         notebookutils.credentials.getSecret with Azure Key Vault")
19     for old, new in REWRITES:
20         line = line.replace(old, new)
21     return line
```

Mechanical rewrites are safe for `fs` and `notebook` calls. Secrets need a design change, so the tool stops and says so.

C5-L1 exercise 5

Hints. (1) What differs between an interactive session and a pipeline run? (2) Inline commands. (3) Move the library to an environment.

Answer. The bug is on line 1.

Fixed version:

```
1 # pandera==0.33.1 is installed in the environment attached to this notebook.
2 import pandera.pandas as pa
3 df = spark.read.table("lh_sales.silver.orders").toPandas()
4 SCHEMA.validate(df, lazy=True)
```

Inline installs are off by default in pipeline runs. Put the library in an environment (Full mode for pipelines), which also pins the version reproducibly.

Common wrong answers.

- Line 2: The import fails only because the install on line 1 did not run.
- Line 3: Reading the table is fine.

C5-L2 Environments, resource profiles and V-Order

C5-L2 exercise 1

Hints. (1) Which resource profile do new workspaces use? (2) What does it do to V-Order? (3) Off, for faster writes.

Answer. b) No. New workspaces default to the writeHeavy profile, with V-Order off. Because: r1) spark.sql.parquet.vorder.default is false in new workspaces to favour write performance.

New workspaces favour writes. Turn V-Order on for tables that are read far more than written.

Common wrong answers.

- a: That was true for older workspaces. New ones default to writeHeavy, with V-Order off.
- c: Files holds non-table data. V-Order applies to Delta table writes.
- d: There is no size threshold. It depends on settings.
- b|r2: Right answer, wrong reason. V-Order still exists. You turn it on where reads matter.

C5-L2 exercise 2

Hints. (1) Which option is reproducible? (2) Which mode does Microsoft recommend for pipelines? (3) Full mode.

Answer. b) An environment in Full mode, attached to the notebooks or set as workspace default Because: r1) Full mode builds a reproducible snapshot, and Microsoft recommends it for pipelines and scheduled runs.

A Full mode environment pins the dependency snapshot for every scheduled run.

Common wrong answers.

- a: Inline installs are turned off by default in pipeline runs, and resolve dependencies differently from run to run.
- c: Quick mode installs at every session start, adding minutes, and suits development more than scheduled runs.
- d: That is an inline install by another route: session-scoped and hard to reproduce.
- b|r2: Right answer, wrong reason. Both modes install the same packages. Full mode is reproducible.

C5-L2 exercise 3

Hints. (1) The table-level property starts with delta. (2) delta.parquet.vorder.enabled. (3) Set it to true.

Answer.

```
1 ALTER TABLE lh.gold.sales_by_region
2 SET TBLPROPERTIES ("delta.parquet.vorder.enabled" = "true");
```

The table property applies V-Order to future writes to this table only.

Common wrong answers.

- g1 = spark.sql.parquet.vorder.enable: That old session setting is removed in Runtime 1.3 and later, and it is not a table property.

C5-L2 exercise 4

Hints. (1) The option belongs on df.write. (2) Its name is parquet.vorder.enabled. (3) Chain mode, option, saveAsTable.

Answer.

```
1 from pyspark.sql import DataFrame
2
3
4 def write_gold(df: DataFrame, table: str) -> None:
5     """Overwrite a read-heavy gold table with V-Order for this write only.
6
7     Args:
8         df: Gold rows.
9         table: Target table, such as lh.gold.sales_by_region.
10    """
11    df.write.mode("overwrite").option("parquet.vorder.enabled",
12                                     "true").saveAsTable(table)
```

A writer option scopes V-Order to one write, so ingestion elsewhere in the session keeps the fast default.

C5-L2 exercise 5

Hints. (1) Reproducible libraries. (2) Most writes are ingestion. (3) Where are reads heavy?

Answer.

- A Full mode default environment gives every scheduled notebook the same pinned libraries.
- writeHeavy suits ingestion, which is most of the Spark work.
- V-Order on gold tables speeds up the reads Power BI makes, without slowing ingestion.

Pin libraries once, optimise for writes by default, and pay for read layout only where reads happen.

Common wrong answers.

- `libs = inline`: Inline installs are off in pipeline runs and are not reproducible.
- `session = read_pbi`: That applies V-Order to every write, including bronze and silver, which pay the write cost for no reader.
- `gold = none_vorder`: Direct Lake reads gold constantly. V-Order is worth its write cost there.

C5-L3 Delta tables shared between Databricks and Fabric

C5-L3 exercise 1

Hints. (1) Which experiences list `TIMESTAMP_NTZ` as supported? (2) Spark does. (3) SQL and Direct Lake do not.

Answer. c) Fabric Spark Runtime 2.0 Because: r1) The interoperability matrix lists `TIMESTAMP_NTZ` as supported in Spark but not in the SQL endpoint or Direct Lake.

Use `TIMESTAMP` for columns that SQL or Power BI users need.

Common wrong answers.

- a: The SQL analytics endpoint does not support `TIMESTAMP_NTZ`.
- b: Direct Lake does not support `TIMESTAMP_NTZ`.
- d: Support differs by experience. Check the matrix.

- c|r2: Right answer, wrong reason. Every listed experience reads Delta. This feature is the issue.

C5-L3 exercise 2

Hints. (1) Which table feature does Runtime 2.0 add by default? (2) Deletion vectors. (3) Which reader lacks them?

Answer. b) A Python notebook (non-Spark) using the default Delta reader
Because: r1) Runtime 2.0 writes deletion vectors by default, and Python notebooks do not support deletion vectors.

Runtime 2.0 writes reader 3, writer 7 with deletion vectors. Python notebooks do not support them.

Common wrong answers.

- a: The SQL analytics endpoint supports deletion vectors.
- c: Direct Lake supports deletion vectors.
- d: The writer's own runtime supports everything it writes.
- b|r2: Right answer, wrong reason. Python notebooks read OneLake. Deletion vectors are the problem.

C5-L3 exercise 3

Hints. (1) The Delta command that returns one row of table details. (2) DESCRIBE ... (3) DESCRIBE.

Answer.

```
1 DESCRIBE DETAIL gold.events;
```

DESCRIBE DETAIL returns minReaderVersion, minWriterVersion and the table features.

Common wrong answers.

- g1 = SHOW: SHOW TBLPROPERTIES lists properties. DESCRIBE DETAIL shows protocol versions and table features.

C5-L3 exercise 4

Hints. (1) Store each reader's supported set. (2) Set difference gives what is missing. (3) Index the dict so unknown readers raise KeyError.

Answer.

```
1 ALL_FEATURES = frozenset(
2     {"deletion_vectors", "column_mapping_name", "column_mapping_id",
3     "timestamp_ntz", "liquid_clustering"}
4 )
5 SUPPORTED: dict[str, frozenset[str]] = {
```

```

5     "spark_2_0": ALL_FEATURES,
6     "sql_endpoint": ALL_FEATURES - {"column_mapping_id", "timestamp_ntz"},
7     "direct_lake": ALL_FEATURES - {"timestamp_ntz"},
8     "python_notebook": frozenset({"timestamp_ntz"}),
9 }
10
11
12 def unsupported(reader: str, features: set[str]) -> list[str]:
13     """Table features a Fabric reader does not support.
14
15     Raises:
16         KeyError: For an unknown reader.
17     """
18     return sorted(features - SUPPORTED[reader])

```

Encoding the matrix lets a release check refuse to publish a gold table with features its readers cannot handle.

C5-L3 exercise 5

Hints. (1) Which readers need this table? (2) Which features do they lack? (3) Check before release.

Answer.

- `TIMESTAMP` is supported by both readers. `TIMESTAMP_NTZ` is not.
- Column mapping in `id` mode is not supported by the SQL endpoint.
- A feature check before release catches changes a Databricks upgrade introduces.

Design for the readers that matter and check features on every release.

Common wrong answers.

- `time = ntz`: Neither the SQL endpoint nor Direct Lake supports `TIMESTAMP_NTZ`.
- `mapping = id_mode`: The SQL analytics endpoint does not support column mapping in `id` mode.
- `gate = trust`: Open format does not mean every reader supports every feature.

C6-L1 Star schemas for semantic models

C6-L1 exercise 1

Hints. (1) What does one row mean? (2) Which columns say which entities it concerns? (3) The dimension keys.

Answer. b) The dimension keys it holds, and their values Because: r1) Dimension key columns set the dimensionality, and their values set the granularity.

One row per combination of the dimension keys: that is the grain.

Common wrong answers.

- a: Measures are the numbers at that grain. They do not define it.
- c: Row count follows from the grain and the data. It does not define it.
- d: Import, DirectQuery or Direct Lake changes how data is read, not the grain.
- b|r2: Right answer, wrong reason. Grain comes from the data design.

C6-L1 exercise 2

Hints. (1) Which way should filters flow in a star? (2) Dimension to fact. (3) Bi-directional breaks that.

Answer. b) Minimise them. They can hurt performance and confuse report users. Because: r1) Bi-directional filters can slow queries and filter in ways users do not expect.

Single-direction one-to-many relationships keep filtering predictable.

Common wrong answers.

- a: The guidance is the opposite: minimise them.
- c: Date tables filter facts one way, like other dimensions.
- d: The concern is behaviour and performance, not support.
- b|r2: Right answer, wrong reason. They still exist. Use them sparingly and on purpose.

C6-L1 exercise 3

Hints. (1) What must be decided first? (2) Dimensions provide keys to facts. (3) Relationships before measures.

Answer.

1. State the fact table's grain
2. Build dimension tables with unique keys in gold
3. Build the fact table with dimension keys and measure columns
4. Create one-to-many, single-direction relationships
5. Write DAX measures

Grain, dimensions, facts, relationships, then measures.

Common wrong answers.

- Decide the grain before you build the fact table, or you will rebuild it.
- Measures depend on relationships to filter correctly.

C6-L1 exercise 4

Hints. (1) Loop over relationships and build a name. (2) Two independent checks, not elif. (3) Append in the order given.

Answer.

```
1 def model_warnings(relationships: list[dict[str, str]]) -> list[str]:
2     """Warnings for relationships that break star-schema guidance.
3
4     Args:
5         relationships: Dicts with from_table, to_table, cardinality and
6             direction.
7
8     Returns:
9         Warning strings in input order.
10    """
11    warnings = []
12    for rel in relationships:
13        name = f"{rel['from_table']}-{rel['to_table']}"
14        if rel["cardinality"] == "many_to_many":
15            warnings.append(f"{name}: many-to-many")
16        if rel["direction"] == "both":
17            warnings.append(f"{name}: bi-directional")
18    return warnings
```

Independent checks report every problem. A review script like this runs on models exported with tools such as Tabular Editor.

C6-L1 exercise 5

Hints. (1) What does Direct Lake on SQL support? (2) What does one row mean? (3) Which way should filters flow?

Answer.

- Compute age_band upstream: Direct Lake on SQL endpoints does not support calculated columns.
- One row per respondent, brand and wave gives a clean fact grain.
- Single-direction relationships keep filters predictable.

Do the modelling work in gold, keep the grain clean, and filter one way.

Common wrong answers.

- where = calc_col: Calculated columns are not supported in Direct Lake on SQL analytics endpoints.

- `grain = resp`: A list column mixes grains and cannot be filtered by brand.
- `rel = both`: Bi-directional filtering is rarely needed and can inflate results.

C6-L2 DAX measures that state their filter context

C6-L2 exercise 1

Hints. (1) What was DIVIDE designed for? (2) What is returned without an alternate result? (3) BLANK.

Answer. b) BLANK Because: r1) DIVIDE returns BLANK when the denominator is zero or BLANK, unless an alternate result is given.

BLANK keeps empty cells empty in visuals.

Common wrong answers.

- a: DIVIDE was designed to handle division by zero without an error.
- c: Only if you pass 0 as the alternate result.
- d: That is what the / operator can return. DIVIDE returns BLANK.
- b|r2: Right answer, wrong reason. The / operator does divide by zero. DIVIDE handles it.

C6-L2 exercise 2

Hints. (1) Which table does ALL name? (2) Filters from dimensions reach the fact table. (3) Remove only the brand filter.

Answer. b) Every filter on `fact_awareness`, including wave and region
Because: r1) ALL on the fact table removes all its filters, including those that reach it through dimensions.

Use `ALL(dim_brand)` to compare against all brands within the same wave and region.

Common wrong answers.

- a: That is `ALL(dim_brand)`. `ALL(fact_awareness)` removes much more.
- c: ALL changes filter context for every filter on the table, not just slicers.
- d: Filter context comes from the visual and the page. ALL removes the table's filters wherever they come from.
- b|r2: Right answer, wrong reason. ALL removes filters from the table or column you give it.

C6-L2 exercise 3

Hints. (1) The DAX keyword for a variable. (2) The safe division function. (3) VAR and DIVIDE.

Answer.

```
1 Weighted aware % =  
2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *  
    fact_awareness[weight] )  
3 VAR base_w = SUM ( fact_awareness[weight] )  
4 RETURN DIVIDE ( aware_w, base_w )
```

Variables name each part. DIVIDE handles a zero base.

Common wrong answers.

- g1 = LET: DAX variables use VAR ... RETURN.

C6-L2 exercise 4

Hints. (1) Two variables, then RETURN. (2) ALL on the brand dimension only. (3) DIVIDE the two.

Answer.

```
1 Share of aware across brands =  
2 VAR this_brand = [Weighted aware %]  
3 VAR all_brands = CALCULATE ( [Weighted aware %], ALL ( dim_brand ) )  
4 RETURN DIVIDE ( this_brand, all_brands )
```

Removing only the brand filter keeps the comparison inside the same wave and region, and DIVIDE handles an empty denominator.

C6-L2 exercise 5

Hints. (1) Does the value depend on the visual's filters? (2) Is it a property of one row? (3) Calculated columns are not an option here.

Answer.

- Age band is a property of each respondent: compute it in gold.
- Percentages depend on the visual's filters: measures.
- The base count also depends on filters: a measure, shown beside every percentage.

Row properties in gold, filter-dependent arithmetic in measures.

Common wrong answers.

- ageband = calc_col: Direct Lake on SQL analytics endpoints does not support calculated columns.

- `share = gold`: A stored percentage cannot re-aggregate correctly when users filter or group differently.
- `base = gold`: The base changes with every filter. It must be computed at query time.

C6-L3 Direct Lake, framing and the fallback you did not plan

C6-L3 exercise 1

Hints. (1) Does Direct Lake copy data? (2) What does it need to know about the tables? (3) Which version to read.

Answer. b) Frames the model: records which Delta table versions queries will use. Because: r1) Framing copies metadata only, takes seconds, and sets the point in time queries see.

Framing takes seconds and fixes the table versions queries see until the next frame.

Common wrong answers.

- `a`: Import copies data. Direct Lake refresh copies metadata only.
- `c`: The model reads Delta tables. It never writes them.
- `d`: Refresh does not change storage mode.
- `b|r2`: Right answer, wrong reason. They do refresh. The refresh is framing.

C6-L3 exercise 2

Hints. (1) Can a view be read as Delta files? (2) What does Automatic do when direct loading fails? (3) Fallback.

Answer. b) The query falls back to DirectQuery and runs more slowly. Because: r1) Views cannot be loaded directly from Delta, and Automatic falls back silently.

Automatic keeps the report working through DirectQuery, silently. Replace views with Delta tables.

Common wrong answers.

- `a`: That is DirectLakeOnly, or Direct Lake on OneLake, which never falls back.
- `c`: Fabric does not materialise views for you. Write a Delta table instead.
- `d`: A view has no Delta files to read.
- `b|r2`: Right answer, wrong reason. Fallback is slower.

C6-L3 exercise 3

Hints. (1) Which option errors when Direct Lake cannot be used? (2) Which is the default? (3) `DirectLakeOnly` and `Automatic`.

Answer.

```
1 Development model: Direct Lake behavior = DirectLakeOnly
2 Production model: Direct Lake behavior = Automatic
```

`DirectLakeOnly` exposes fallback causes during development. `Automatic` protects users in production.

Common wrong answers.

- `g1 = DirectQueryOnly`: `DirectQueryOnly` always uses `DirectQuery`. It measures fallback cost but does not reveal where fallback happens.

C6-L3 exercise 4

Hints. (1) A dict from SKU to (files, rows). (2) Strictly greater than the limit is a problem. (3) Index the dict so unknown SKUs raise `KeyError`.

Answer.

```
1 SMALL = (1000, 300)
2 GUARDRAILS: dict[str, tuple[int, float]] = {
3     "F2": SMALL, "F4": SMALL, "F8": SMALL, "F16": SMALL, "F32": SMALL,
4     "F64": (5000, 1500), "F128": (5000, 3000), "F256": (5000, 6000),
5 }
6
7
8 def guardrail_problems(sku: str, parquet_files: int, rows_millions:
9     float) -> list[str]:
10     """Direct Lake per-table guardrails a table exceeds on a SKU.
11
12     Raises:
13         KeyError: For a SKU not in the table.
14     """
15     max_files, max_rows = GUARDRAILS[sku]
16     problems = []
17     if parquet_files > max_files:
18         problems.append("files")
19     if rows_millions > max_rows:
20         problems.append("rows")
21     return problems
```

A nightly check against the guardrails catches small-file build-up before Direct Lake falls back or framing fails.

C6-L3 exercise 5

Hints. (1) How many files does each append add? (2) What merges small files? (3) `OPTIMIZE` after the write.

Answer. The bug is on line 3.

Fixed version:

```
1 from pyspark.sql import DataFrame
2
3
4 def publish_hourly(df: DataFrame, table: str) -> None:
5     """Append the latest hour, then compact small files for Direct
        Lake."""
6     df.repartition(1).write.mode("append").saveAsTable(table)
7     df.sparkSession.sql(f"OPTIMIZE {table}")
8     return None
```

Hourly appends add a file each hour. OPTIMIZE compacts them, keeping the table under the guardrail. On busy tables, run it on a schedule rather than every hour.

Common wrong answers.

- Line 2: The docstring is fine.
- Line 4: Returning None is harmless.